

➤ Imperative commands (single line commands to create the objects)

⌘ **kubectl run my-pod --image=nginx** [here *run* is used, not *create*]

⌘ **restartPolicy:** always

always: restart pod even if the container exits with exit code 0

onFailure: restart pod only if container's exit code is non-zero

never: never restart the pod even if container stops.

⌘ **namespace:** default

it is *logical* grouping of a cluster to separate logically like dev, prod, ..etc.

You can set the resource allocation according to namespace as well.

⌘ **labels:** *run: my-pod*

⌘ **port:** not exposed

[port is never exposed from container though, means from Service you can route to any port of the pod you want; *port* only there for ease of querying pods according to port or to display the usable port of the container to user when they get *kubectl describe pod <pod name>*]

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
    name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

(default configs of the command)

⌘ **kubectl create deployment my-deploy --image=nginx** [here *create* is used]

⌘ Deployment name: my-deploy

⌘ ReplicaSet: my-deploy-*<hash>*

⌘ Pod: my-deploy-*<hash>*-*<random>*

⌘ labels: *app:my-deploy*

- These are the things kubectl create commands can create. pod cannot be created using kubectl create command

clusterrole	Create a cluster role
clusterrolebinding	Create a cluster role binding for a particular cluster role
configmap	Create a config map from a local file, directory or literal value
cronjob	Create a cron job with the specified name
deployment	Create a deployment with the specified name
ingress	Create an ingress with the specified name
job	Create a job with the specified name
namespace	Create a namespace with the specified name
poddisruptionbudget	Create a pod disruption budget with the specified name
priorityclass	Create a priority class with the specified name
quota	Create a quota with the specified name
role	Create a role with single rule
rolebinding	Create a role binding for a particular role or cluster role
secret	Create a secret using a specified subcommand
service	Create a service using a specified subcommand
serviceaccount	Create a service account with the specified name
token	Request a service account token

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-deployment
  name: my-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-deployment
  strategy: {}
  template:
    metadata:
      labels:
        app: my-deployment
    spec:
      containers:
        - image: nginx
          name: nginx
          resources: {}
status: {}
```

(default configs of the command)

labels are automatically added.

- **ReplicaSet** cannot be created directly using imperative way, because Deployment is there to create and manage replicasets.
- **kubectl expose** takes replication controller, service, deployment, or pod and creates a *Service* to expose that.

```
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                  ClusterIP           10.96.0.1       <none>           443/TCP          36m

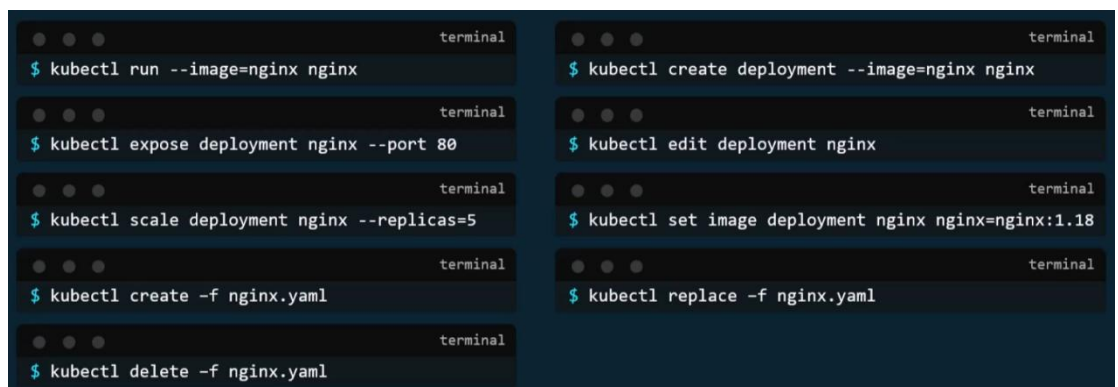
NAME      READY   STATUS    RESTARTS   AGE
pod/my-pod 1/1     Running   0          34s
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                  ClusterIP           10.96.0.1       <none>           443/TCP          37m
service/my-pod                      ClusterIP           10.96.68.232    <none>           80/TCP           6s

NAME      READY   STATUS    RESTARTS   AGE
pod/my-pod 1/1     Running   0          76s
```

you can see here, It created a service named *my-pod*. Its of type *ClusterIP* which is default.

- If you give *--type=NodePort* flag, then it'll create a service of type NodePort

```
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80 --type NodePort
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                  ClusterIP           10.96.0.1       <none>           443/TCP          39m
service/my-pod                      NodePort            10.96.119.232   <none>           80:32530/TCP     3s
```



The image shows a grid of terminal windows demonstrating various kubectl commands for managing a deployment named 'nginx':

- `$ kubectl run --image=nginx nginx`
- `$ kubectl expose deployment nginx --port 80`
- `$ kubectl scale deployment nginx --replicas=5`
- `$ kubectl create -f nginx.yaml`
- `$ kubectl delete -f nginx.yaml`
- `$ kubectl create deployment --image=nginx nginx`
- `$ kubectl edit deployment nginx`
- `$ kubectl set image deployment nginx nginx=nginx:1.18`
- `$ kubectl replace -f nginx.yaml`

[these are the imperative commands]

create -f is imperative because if already the same config present it'll fail, where as *apply -f* will not fail.

- Important trick

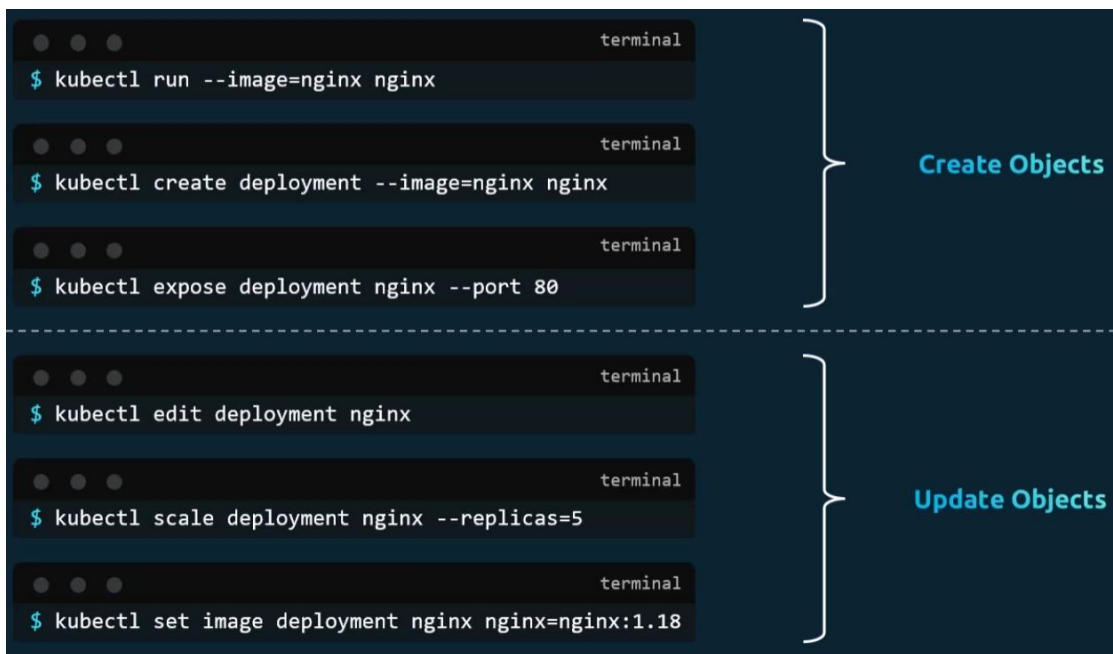
- `kubectl create deployment my-deploy --image=nginx --dry-run=client -o yaml`

`kubectl run my-pod --image=nginx --dry-run=client -o yaml`

- it'll give the yaml file of default deployment, remove the unnecessary fields, customize that, and create deployment
- Because of the flag *--dry-run=client* it'll not create the deployment, just simulate and give the output as yaml.

➤ **kubectl get pod my-pod -o yaml**

- It'll give the running config of the pod as an yaml.
- Can be done with any types of objects i.e. pod, replica set, container runtime, deployment ..etc
- But it'll contain a lot of things, that needs to be removed.



➤ **edit vs replace command**

- *kubectl edit* command open the running status of the object (pod, replicaset, replication controller, deployment, service ..etc) in a yaml file.

This configurations are there in the memory, not stored in the yaml file that was being used to create the object.

So, if you do *kubectl edit* then it'll not have any track of the changes. If someone later want to update the object, and he sees the yaml file present (not the running configuration yaml) and try to update, it might do some wrong changes in the running object.

- *kubectl replace* command uses the physical yaml file to update the running object.

It does *dumb* update in the running object. Whatever there in the physical yaml, it'll update all the things and delete the configs which are not

present in that physical yaml file.

By default it doesn't delete the current object (if anything doesn't match with physical yaml), to replace force fully you can give the flag *--force*.

```
vagrant@controlplane:~$ kubectl replace --force -f pd.yaml
pod "my-pod" deleted from default namespace
pod/my-pod replaced
```

It looks similar to *kubectl apply*, but *kubectl apply* does *intelligent* update. It finds the *diff* between running object and the physical yaml (used to update the object) and update those only.

It *doesn't delete any configs* which are not present in the current yaml but there in the running object.

kubectl apply is *declarative* command.

- If there are multiple files out of which objects will have to be created, then just give the path of the directory where those are present to *kubectl apply*, it'll create all those.

```
kubectl apply -f <path where all files are present>
```

--- kubectl apply internals ---

➤ [here I'll take an example of a deployment named *my-dep*]

➤ There are 3 things.

yaml (file that you write)

last applied config (when you execute `kubectl get deploy my-dep -o yaml`) **[JSON format]**

live config (when you execute `kubectl describe deploy my-dep`)

[*deploy* is shorthand of *deployment*]

⌘ Whenever you do **declarative** change using a *yaml* i.e.

`kubectl apply -f my-dep.yaml`

it changes both *live config*, *last applied config*. and obviously *yaml* is changed by you only.

⌘ Whenever you do **imperative** change using the command i.e.

`kubectl scale deployment my-dep --replicas=5`

it changes only *live config*. *Last applied config* will not be changed. And of course, *yaml* has not been changed.

command type	yaml	last applied config	live config
declarative	Y	Y	Y
imperative	N	N	Y

➤ There are a few commands [*deployment is used for example; can be used for any objects*]:

⌘ **kubectl diff -f <yaml file name>**

compare the *yaml* with the *live config* along with *last applied* and display how it'll look like after you execute `kubectl apply`.

⌘ **kubectl apply view-last-updated deployment <deployment name>**

show the last updated configs [which is present actually in JSON] in *yaml* format cleanly.

⌘ **kubectl get deployment <deployment name> -o yaml**

kubectl get -f <deployment yaml file name> -o yaml [it fetch from the file]

Both will display the *live configs* of the *deployment* object.

Also, there will be a JSON structure under *annotations* field where you can see *last updated config*.

➤ Stage-1: [create a deployment from yaml]

⌘ Initial yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.16.1
          ports:
            - containerPort: 80
```

⌘ **kubectldiff -f <yaml file name>** will display like below

```
vagrant@controlplane:~$ kubectldiff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-d
--- /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1106384729/apps.v1.Deployment.default.ng-dep
@@ -0,0 +1,41 @@
+apiVersion: apps/v1
+kind: Deployment
+metadata:
+  creationTimestamp: "2026-04-30T21:32:08Z"
+  generation: 1
+  name: ng-dep
+  namespace: default
```

[cropped]

⌘ Run the deployment using: *kubectl apply -f <yaml file name>*

- ⌘ **kubectl apply view-last-applied deployment <deployment name>**

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

- Stage 2: [change image using imperative command]

- ⌘ **kubectl set image deploy <deployment name> <container name>=<new image>**

kubectl set image deploy ng-dep nginx=nginx:1.14.2

```
vagrant@controlplane:~$ kubectl set image deploy ng-dep nginx=nginx:1.14.2
deployment.apps/ng-dep image updated
```

- ⌘ **kubectl diff -f <deployment file name>**

```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-dep
--- /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1145231250/apps.v1.Deployment.default.ng-dep
@@ -6,7 +6,7 @@
   kubectl.kubernetes.io/last-applied-configuration: |
     {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"name":"ng-dep","namespace":"default","resourceVersion":"33813"},"spec":{"replicas":3,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"},"spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx","ports":[{"containerPort":80}]}]}}}}}
-   generation: 2
+   generation: 3
    name: ng-dep
    namespace: default
    resourceVersion: "33813"
@@ -29,7 +29,7 @@
    app: nginx
    spec:
      containers:
-       - image: nginx:1.14.2
+       - image: nginx:1.16.1
      imagePullPolicy: IfNotPresent
```

[you can see, yaml file has *nginx:1.16.1* but live config has *nginx:1.14.2*]

[so, its showing - on *nginx:1.14.2* and + on *nginx:1.16.1*]

⚡ **kubectl apply view-last-applied deploy <deployment name>**

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[its showing image as `nginx:1.16.1` because imperative change doesn't effect last applied config]

➤ Stage-3: [update yaml, remove replica field, then declarative update]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-dep
--- /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-2827279498/apps.v1.Deployment.default.ng-dep
@@ -6,14 +6,14 @@
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"name":"ng-dep","namespace":"default","resourceVersion":"33813","uid":"92189358-0dd5-4f8f-8144-1936af5322ab"},"spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"},"spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx","ports":[{"containerPort":80}]}]}}}}}
- generation: 2
+ generation: 3
  name: ng-dep
  namespace: default
  resourceVersion: "33813"
  uid: 92189358-0dd5-4f8f-8144-1936af5322ab
  spec:
    progressDeadlineSeconds: 600
- replicas: 3
+ replicas: 1
    revisionHistoryLimit: 10
    selector:
      matchLabels:
@@ -29,7 +29,7 @@
      app: nginx
    spec:
      containers:
-       - image: nginx:1.14.2
+       - image: nginx:1.16.1
```

[its showing if I update, *replica* and *image* will be updated]

- Now in last applied config, *replicas* field would have been removed because we deleted that field in yaml.

```
vagrant@controlplane:~$ kubectl apply view-last-applied -f simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
-       - image: nginx:1.16.1
        name: nginx
        ports:
-       - containerPort: 80
```

[replicas is not there, image is *nginx:1.16.1*]

^ *live config*

```
spec:
  progressDeadlineSeconds: 600
  replicas: 1
```

```
spec:
  containers:
  - image: nginx:1.16.1
    imagePullPolicy: IfNotPresent
```

[the image and replicas got changed]

➤ Its looking like, why there is need of *last applied config* because when we apply, the live config is becoming same as the yaml.

➤ Stage-1: [create declarative from yaml]

^ See the yaml now

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

[added one label only]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[last applied config will be same as yaml because of declarative change]

- Stage-2: [add one more label in imperative way; *kubectl edit* command]

```
Labels:
  app: ng-deployment
  keyx: valx
```

[live config changed]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
```

[last applied config will be same because it was imperative update]

- Stage-3: [update the replicas using declarative]

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  replicas: 3
  selector:
```

[updated yaml

file]

```

creationTimestamp: "2026-04-01T16:07:16Z"
- generation: 1
+ generation: 2
  labels:
    app: ng-deployment
    keyx: valx
@@ -16,7 +16,7 @@
  uid: 5071163c-87e2-49db-a1a0-000000000000
  spec:
    progressDeadlineSeconds: 600
-   replicas: 1
+   replicas: 3
  revisionHistoryLimit: 10

```

[ignore generation]

[`kubectl diff -f <deployment file name>`]

you can see, its not showing *keyx: valx* label to be deleted when it is not there in our current yaml file.

➤ `kubectl apply -f <deployment file name>`

```

vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
    name: ng-dep
    namespace: default
spec:
  replicas: 3

```

[last applied config]

```

creationTimestamp: "2026-04-01T16:07:16Z"
generation: 2
labels:
  app: ng-deployment
  keyx: valx
name: ng-dep
namespace: default
resourceVersion: "37619"
uid: 5071163c-87e2-49db-a1a0-000000000000
spec:
  progressDeadlineSeconds: 600
  replicas: 3

```

[live config, replicas got increased but that label (keyx: valx) is still there]

➤ So, how does *kubectl apply* work then?

➤ It does the below

Sets fields, that appear in the configuration (yaml) file, in the live configuration.

Clears fields, that removed from the configuration (yml) file, in the live configuration.

- It does the above after comparing with *last applied field*.
- If any field is there in [either of *yml* or *last applied config*] (or) [the field is present in both *yml* and *last applied config*] then that field will be updated in the *live config*.
- In the previous example. The label (keyx: valx) was not there in both *yml* and *last applied config*, so it didn't track that.

➤ In Simple Words it does the below

It tracks the configs which are present in either *yml* or *last applied config* and updates the *live config* accordingly. If something is there in *live config* but not there in both *yml* and *last applied config* then it'll skip that.

Merging changes to primitive fields

Primitive fields are replaced or cleared.

Note: - is used for "not applicable" because the value is not used.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

Merging changes to map fields

Fields that represent maps are merged by comparing each of the subfields or elements of the map:

Note: - is used for "not applicable" because the value is not used.

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

yaml	last applied config	live config	RES
Y	Y	Y/N	yaml : Y last : update to yaml live : update to yaml
Y	N	Y/N	yaml : Y last : update to yaml live : update to yaml
N	Y	Y/N	yaml : N last : remove live : remove
N	N	Y	yaml : N last : N live : no update

Namespaces

- Consider the below analogy



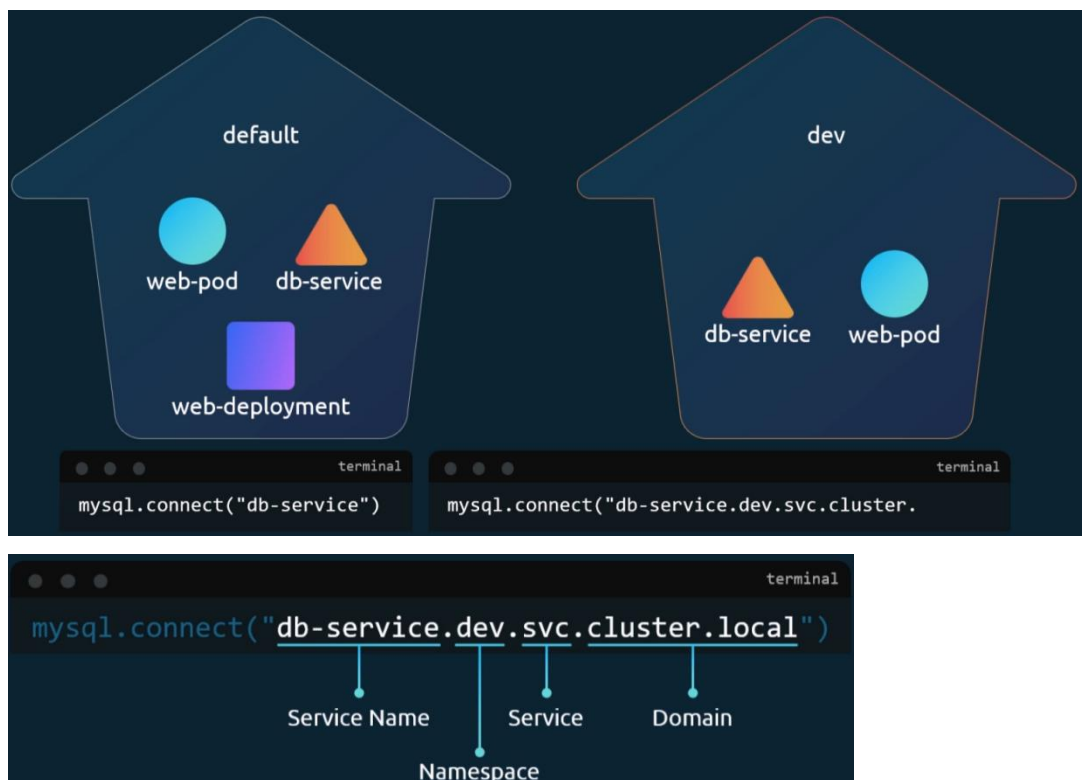
- ⌘ There are 2 houses,
 - In one house (house-A), one person called Mark *Smith* is there.
 - In another house (house-B), one person called Mark *Williams* is there.
 - ⌘ In house-A, people can call Mark *Smith* as only Mark because there is no other guy having this name in that house.
 - Similarly in house-B, people can call Mark *Williams* as Mark only.
 - ⌘ But, if people of house-A wants to call Mark *Williams* then they need to call by full-name i.e. Mark *Williams*. Not only Mark.
 - Similarly for people of house-B as well. They need to call Mark *Smith*.
 - ⌘ A person who neither belong to house-A nor house-B, needs to call those by their full name only i.e. Mark *Smith* or Mark *Williams*.
 - ⌘ Consider these **houses as namespaces**.
 - Inside a namespace, resources having **same names cannot be present** (you can't create 2 pods having exactly same name). but, in multiple namespaces, resources of **same name can be present**.
 - ⌘ For example, if you want multiple environments like *dev*, *prod* and you don't want to update any *production* pods while testing during *development* or something like that, **namespaces** help in that.
 - You can have exact same resources in both the namespaces.
- Example:
 - ⌘ Consider a single namespace scenario:
 - ⌘ Some pods are running DB. And service name (ClusterIP) is **db**.
 - ⌘ The application pods who want to use database can use **db** directly to access the database.

- Consider a multiple namespace scenario:
 - In the same example as above, let's imagine 2 namespaces are there.
 - Now, the application pods of one namespace can directly use **db** to access the database running in same namespace.
- But, if they want to use **db** of the other namespace then they need to give the full DNS path i.e. **db.<namespace name>** or **db.<namespace name>.svc.cluster.local** [you don't need to give this, **db.<namespace name>** is converted to this internally]

➤ **db.<namespace name>.svc.cluster.local**

in here, **svc** means don't access the pod directly, rather do using the **db** service.

cluster.local is the internal DNS of the cluster. [.local is a **top-level-domain** just like .com, .in ..etc but it is internal]



- If you want to get or create the resources present in a specific namespace then

kubectl get <resource type like pod, rs, deploy> -n <namespace name>

kubectl create -f <yaml file name> -n <namespace name>

[NOTE] You need to write 'create', not 'apply' when you are creating something from the yaml **first time (still have to verify this)**]

[if the **namespace** key is present in the *yaml* file already, then no need to give **--namespace** flag]

- ⌘ Or, in *yaml* you can write *namespace* inside metadata section.

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  namespace: dev
```

- ⌘ To create a *namespace*, the *yaml* file format will be like this

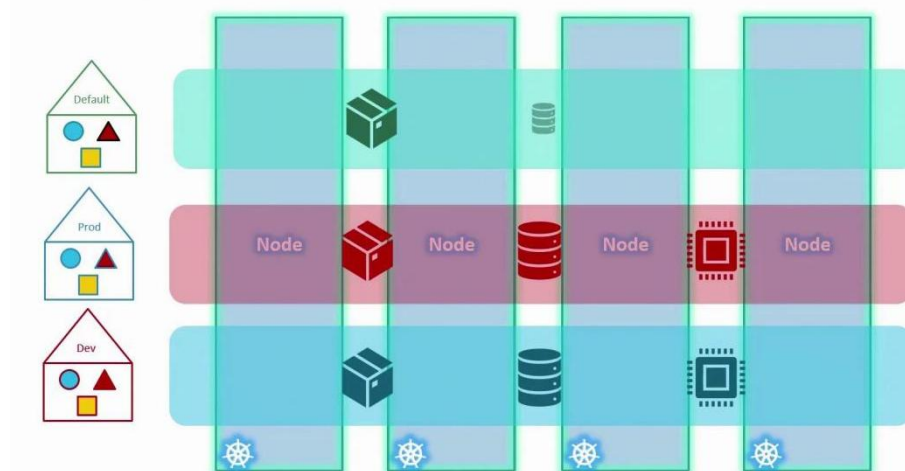
```
vagrant@controlplane:~$ kubectl create namespace test --dry-run=client -o yaml
apiVersion: v1
kind: Namespace
metadata:
  name: test
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

- All the *namespace* configures while setup:
 - ⌘ When a kubernetes *cluster* is first setup, **default** namespace is created. Whatever resources you create, it is created inside this.
 - ⌘ Kubernetes creates a set of Pods and Services for its internal purpose. These all stays in the namespace called **kube-system**.
 - ⌘ There is another namespace called **kube-public** where resources available to all users are created.

- You can create **quota** (long format: **ResourceQuota**) to the namespaces (setting the limits)

Namespace – Resource Limits



- If you want to change the default (not talking about the namespace having name *default*) namespace

```
kubectl config set-context $(kubectl config current-context) --  
namespace=<namespae name which has to be made default>
```

- config -

- The config file path is `~/.kube/config`.

```
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: LS0tLS1CRUdJTiB1c2Vybm9keSB0b290aW50
  server: https://192.168.56.11:6443
  name: kubernetes
contexts:
- context:
  cluster: kubernetes
  namespace: test
  user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
users:
- name: kubernetes-admin
  user:
    client-certificate-data: LS0tLS1CRUdJTiB1c2Vybm9keSB0b290aW50
    client-key-data: LS0tLS1CRUdJTiB1c2Vybm9keSB0b290aW50
```

-

```
contexts:
- context:
  cluster: kubernetes
  user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@k
```

[if namespace is not there]

it looks something like this (its content format is of yaml, but the filename is `config` only, not `config.yaml`)

- It mainly contains three things: **cluster**, **contexts**, **users**
- **clusters**
 - ♣ its a list of clusters.
 - ♣ Each cluster contains *name*, *cluster* (cluster contains *certificate-authority-data*, *server*) fields.
 - ♣ **name**
 - ♣ its just an identifier that will be used later inside *context* (you can see inside *context* field).
 - ♣ every operation i.e. create pod, get svc ..etc **hits this URL**.

kubectl → API Server → Cluster

cluster.server

it's the end-point of *API Server* where all *kubectl* command goes.

2 cluster.certificate-authority-data

- Ensures you're talking to the real cluster
- Prevents man-in-the-middle attacks

➤ users

- It contains the list of users.
- It defines “Who are you when talking to the Cluster”
- Each entries of *user* contain the below fields **name**, **user** (*user* contains *client-certificate-data*, *client-key-data*) fields.
- name**
just a identifier to the user. Which is referenced inside *context* field (you can see in the yaml).
- user.client-certificate-data** & **user.client-key-data** (common in local setup)
certificate is like ID card; *key* is like signature (Analogy)
- In real-world, instead of certificate and key, *token* is used.
- token: eyJhbGciOiJSUzI1NiIsIn**
 - it is used in ServiceAccounts, CI/CD, API access.
- In cloud provider cases, external Auth is present.
- All-In-One, *user* field is for authentication with the *cluster*.**

➤ contexts

- It is something that reference all the things i.e.
`context = cluster + user + default namespace` **--- IMPORTANT ---**
- When you switch to a context, all the things will be switched i.e. user, cluster, default namespace
- When you execute any command like
kubectrl get pod
it is executed like the below
kubectrl get pod --namespace=<default namespace name>
- If the namespace field is not present in the *config* file (refer the image), then it'll take the namespace called "default" by default.
- But, whatever namespace you give (override) like the below
kubectrl get pod --namespace=<namespace name other than context default

one>

the **user** and **cluster** will be same only.

- **NOTE** **namespace** field will not be there inside the *config* file. -----
- You can create as many context you want.
 - ⌘ **Same cluster + different namespaces** : new context
(**optional**, if you want to maintain different contexts; otherwise you can just override the default namespace and move on in the same context using --namespace=<other namespace name>)
 - ⌘ **Same cluster + different user** : new context
(its **mandatory**)
you cannot switch user independently, only you can switch *context*.
 - ⌘ **Different cluster** : new context
(its **mandatory**)
- **kubectl config --help**

```
Available Commands:
current-context      Display the current-context
delete-cluster       Delete the specified cluster from the kubeconfig
delete-context       Delete the specified context from the kubeconfig
delete-user          Delete the specified user from the kubeconfig
get-clusters         Display clusters defined in the kubeconfig
get-contexts         Describe one or many contexts
get-users            Display users defined in the kubeconfig
rename-context       Rename a context from the kubeconfig file
set                  Set an individual value in a kubeconfig file
set-cluster          Set a cluster entry in kubeconfig
set-context          Set a context entry in kubeconfig
set-credentials      Set a user entry in kubeconfig
unset                Unset an individual value in a kubeconfig file
use-context          Set the current-context in a kubeconfig file
view                 Display merged kubeconfig settings or a specified kubeconfig file
```

- ⌘ **use-context** is used to set a new **current-context** (see in yaml).
- If you want to create a new context
kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test

```
vagrant@controlplane:~$ kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test
Context "test-context" created.
vagrant@controlplane:~$ kubectl config get-contexts
CURRENT  NAME              CLUSTER  AUTHINFO  NAMESPACE
*        kubernetes-admin@kubernetes  kubernetes  kubernetes-admin
test-context  kubernetes  kubernetes-admin  test
```

- You can use this command to view all the configs.
kubectl config view

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.56.11:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
```

[cropped, it'll display the whole config

file]

- If you want to change the *current-context* then

kubectl config use-context <context name>

----- THE END OF CONFIG -----

- There is another type of resource called **ResourceQuota** which is used to set the limit of a specific **namespace**.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```



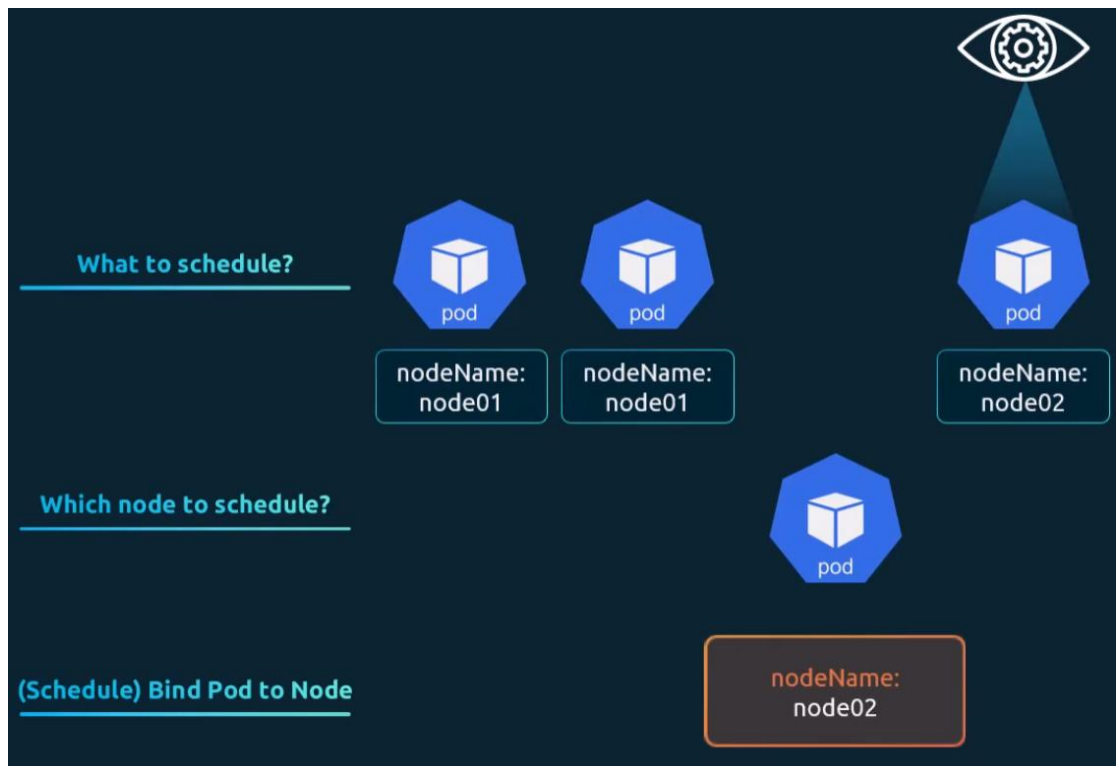
- Scheduling -

- In a running pod, when you see it, there will be a field called **nodeName** will be present, which contains the worker node's name where that specific pod is running.

Just execute `kubectl edit pod <pod name>` to see this.

```
enableServiceLinks: true
nodeName: node02
preemptionPolicy: PreemptLowerPriority
```

- How *scheduling* works?



- When you run a pod (without specifying *nodeName* field), the pod object is created.
- Then, *scheduler* sees this, and assign a node to it (*nodeName*).
NOTE: this **nodeName** field is under *spec* field of *Pod*'s config.
- Then, one **binding** will be created in the target node which will bind the *pod* to the *target node*.
- If the Scheduler doesn't exist,
 - then *nodeName* will not be appended to the pod's config.
 - Even if you give *nodeName*, **binding** object will not be created in the target node unless manually send the POST request to the target worker node to create Binding object.

- ⌘ In this case, the Pod will be in pending state.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node03
```

Every 2.0s: kubectl get pod

NAME	READY	STATUS	RESTARTS	AGE
ng-dep-66b9cb6d4d-4r822	1/1	Running	0	38h
ng-dep-66b9cb6d4d-f4gl4	1/1	Running	0	38h
ng-dep-66b9cb6d4d-f52lt	1/1	Running	0	39h
test-pod	0/1	Pending	0	13s

- ⌘ I simulated is giving a *nodeName* which doesn't exist inside */etc/hosts* file.

So, the *Binding* object will not be created.

- ⌘ After sometime, the pod will be **deleted** because its status couldn't be 'Running'.
- Simulating **Manual Scheduling** of a Pod to a Node
 - ⌘ First create a yaml config of the node giving *nodeName* field.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node01
```

- ⌘ Now, create the pod

```
vagrant@controlplane:~$ kubectl create -f test-pod.yaml
pod/test-pod created
```

- ⌘ Now, you need to create a *Binding* yaml file.

```
apiVersion: v1
kind: Binding
metadata:
  name: test-binding
target:
  apiVersion: v1
  kind: Node
  name: node01
```


NOTE: It doesn't contain any data about the Pod, it only contains the details of the target node.

- ⌘ The yaml is just to understand the key value pairs, in real there is **no such long-lived object called *Binding* exists.**

It's just the JSON payload to be sent in the POST request to bind the pod to the target node.

- ⌘ Now, you need to trigger the POST request to the **API Server** to bind the pod to the *target worker node*.

NOTE: the worker node doesn't expose any end-points, only API Server in the master node does this.

And, you need to trigger the request to this API Server's end point.

- ⌘ The POST request looks like this (just an example)

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"n
http://$SERVER/api/v1/namespaces/default/pods/$PODNAME/binding
```

- ⌘ You'll get the \$SERVER from *kubectl config view* command. It'll be in the form

<API server node's IP>:<API Server's port>

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: DATA+OMITTED
  server: https://192.168.56.11:6443
```

- ⌘ \$PODNAME is the Pod's name that you get from *kubectl get pod* command.

```
vagrant@controlplane:~$ kubectl get pod
NAME                                READY   STATUS
ng-dep-66b9cb6d4d-4r822            1/1     Runni
ng-dep-66b9cb6d4d-f4gl4            1/1     Runni
ng-dep-66b9cb6d4d-f52lt            1/1     Runni
test-pod                           1/1     Runni
```

- ⌘ Now, trigger the request

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"name": "test-binding"}, "target": {"apiVersion": "v1", "kind": "Node", "name": "node01"}}' \
  https://192.168.56.11:6443/api/v1/namespaces/default/pods/test-pod/binding/
```

- ⌘ It might not work, because you need to provide token or certificates keys in the URL. **WILL SEE LATER.**

- Labels and Selectors -

- **labels** and **selectors** are used for **grouping** and **filtering**.
- You can give multiple properties inside **labels** which will be helpful in filtering the resources properly.
 - ⌘ You can give as many key value pairs inside **labels** field.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    name: ng-deployment
    key1: val1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
        key2: val2
    spec:
      containers:
```

- ⌘ (here I have given 2 labels to each deployment and pod template)

- ⌘ Then you can select the resources with

--selector key1=val1,key2=val2 (you can give any number of label here)

```
vagrant@controlplane:~$ kubectl get all --selector app=nginx,key2=val2
NAME                                READY   STATUS    RESTARTS   AGE
pod/ng-dep-797565664d-dpkjk        1/1     Running   0           80s
pod/ng-dep-797565664d-qljts        1/1     Running   0           80s
pod/ng-dep-797565664d-zgq4k        1/1     Running   0           80s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/ng-dep-797565664d   3          3         3       80s
```

NOTE: ReplicaSet created by Deployment will have same label as Pods.

- In case of Deployment or ReplicaSet, whatever you write inside **matchLabels** field will be used to get the Pods.
 - ⌘ All the fields inside **matchLabels** should be there inside the Pod's labels.
 - ⌘ Pod's labels can contain either **same or extra** fields than **matchLabels**, but **not less**.

If pod doesn't contain all the fields present in *matchLabels* then it'll not be selected.

➤ Annotations

- ⌘ You can also give a field **annotations** inside *metadata* field.
- ⌘ Annotations may include *build version, contact information, or other integration data*.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: simple-webapp
  labels:
    app: App1
    function: Front-end
  annotations:
    buildversion: "1.34"
```

- ⌘ Annotations are ideal for *adding tool details, version information, or contact data* without impacting how the system groups or selects objects



--- Taints and Tolerations ---

- Real-life analogy (Assumptions HIT kills cockroaches, but cannot kill Lizards):
 - ♣ You want to prevent cockroaches on your bed. So you sprayed HIT around it.
 - ♣ Now, Cockroaches cannot come because they are *intolerant* to HIT.
 - ♣ But, Lizards might come because HIT cannot kill them.
Or, in other words, lizards are *tolerant* to HIT.
- So, How these concepts being used?
 - ♣ You put *taint* (HIT in the analogy) on the *nodes*, to prevent the *Pods* being scheduled to it.
 - ♣ Only the *tolerant* pods (lizard in the analogy) can be scheduled to the *tainted node*.
 - ♣ *Nodes* are *tainted* to prevent the *Pods* to be scheduled to them.
Pods are being *tolerated* (when required) to tolerate the *tainted nodes*.
 - ♣ In simple words,
Taints on nodes **repel pods** that do not have **matching tolerations**.
Tolerations in pods allow them to **overcome the taint** and be **scheduled on that node**.

➤ Taint the Node

kubectl taint nodes <node name> **key=value:**<taint-effect>

[always try to use plural forms i.e. **nodes** not **node**]

Ex: kubectl taint nodes node01 env=prod:NoSchedule

- ♣ to remove the taint of the node, just put a - at the end.

kubectl taint nodes <node name> **key=value:**<taint-effect>-

Ex: kubectl taint nodes node01 env=prod:NoSchedule-

- ♣ Example:

- ♣ Initially no *taint* was there.

```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints:                <none>
```

- ♣ taint node01

```
vagrant@controlplane:~$ kubectl taint nodes node01 env=prod:NoSchedule
node/node01 tainted
```

- node01 got tainted.

```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints: env=prod:NoSchedule
```

- Remove the taint

```
vagrant@controlplane:~$ kubectl taint nodes node01 env=prod:NoSchedule-
node/node01 untainted
```

- Taint got removed from node01

```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints: <none>
```

- `kubectl taint nodes <node name> key=value:<taint-effect>-`

it is to remove the taint when the exact match will be there.

- You can also remove the taints for *partial change* by giving this

`kubectl taint nodes <node name> key-`

it'll remove the taint of the node where the key is matched.

- When there are multiple taints, it'll be seen like this

```
Taints: app=product:NoSchedule
env=dev:NoSchedule
```

➤ Tolerate the Pod

- To tolerate a pod, there is no *imperative* command, also you **cannot** update using `kubectl edit pod <pod name>` command.

you need to configure the yaml.

- Inside the Pod's *spec* field, you can write *tolerations*

```
spec:
  tolerations:
  - key: "env"
    operator: "Equal"
    value: "prod"
    effect: "NoSchedule"
```



[**double-quote** is mandatory]

- Operator: *"Equal"*

because while giving taint, we wrote *env=prod*

- You can see, when you create a cluster and add master and worker nodes, pods will not be scheduled to master node.

It is because **master node has a taint**

```
Taints: node-role.kubernetes.io/control-plane:NoSchedule
```

and pods don't have *tolation* to this *taint*.

➤ Taint Effects

➤ **NoSchedule** (we used this previously)

- ⌘ New Pods will **not be scheduled** on this node *if they are **not having the toleration***.
- ⌘ Existing Pods will **remain as it is**. (*even if they doesn't have the toleration*)
- ⌘ In simple terms:
New Pods: ❌
Existing Pods: ✅
- ⌘ In our analogy, you put HIT around of the bed. Cockroaches who are already on the bed will survive but no outside cockroaches can come.

➤ **PreferNoSchedule**

- ⌘ Kubernetes will try to avoid this node to schedule the pods.
- ⌘ But, it might schedule if there is no other option.
- ⌘ In simple terms:
New Pods: ❌✅
Existing Pods: ✅

➤ **NoExecute**

- ⌘ New Pods will not be scheduled to this node (same as *NoSchedule* effect)
- ⌘ Existing Pods will be removed (different from *NoSchedule* effect)
- ⌘ In simple terms:
New Pods: ❌
Existing Pods: ❌
- ⌘ In our analogy, you put HIT around the bed and also on the bed. Now cockroaches will be dead and no outside cockroaches can come. Lizards are safe.

➤ NOTE: All the not-scheduling and delete from the node things which are written above, those are for the pods who don't have *tolerance* to that *taint*.

If the pods are having *tolerance*, then they (new and existing) will not be affected.

⌘ -----

--- nodeSelector and nodeAffinity ---

- While *taint* and *tolerations* were to avoid a node to be scheduled for pods, *nodeSelector* and *nodeAffinity* are there to schedule a node for the pods.

➤ nodeSelector

- It works in the same principle *selector*.
- First provide the *labels to the node*. [before we labelling the Pods]

```
kubectl label nodes node1 env=prod
```

- Use *nodeSelector* field to match the Node labels. [ReplicaSet's *selectors* field]

```
spec:
  nodeSelector:
    env: prod
```

- Example

- Created a 2 pod yaml files (one for env=prod, another for env=dev)

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: prod-pod
    name: prod-pod
spec:
  nodeSelector:
    env: prod
  containers:
  - image: nginx
    name: prod-cont
    ports:
    - containerPort: 80
      name: nginx-port
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: dev-pod
    name: dev-pod
spec:
  nodeSelector:
    env: dev
  containers:
  - image: nginx
    name: dev-cont
    ports:
    - containerPort: 80
      name: nginx-port
```

- Label *node01* as *env=dev*, and *node02* as *env=prod*

```
vagrant@controlplane:~$ kubectl label nodes node01 env=dev
node/node01 labeled
vagrant@controlplane:~$ kubectl label nodes node02 env=prod
node/node02 labeled
```

```
Labels:          beta.kubernetes.io/arch=amd64
                beta.kubernetes.io/os=linux
                env=dev
```

[node01]

```
Labels:          beta.kubernetes.io/arch=amd64
                beta.kubernetes.io/os=linux
                env=prod
```

[node02]

- Created prod pod and it was scheduled in node02

```
vagrant@controlplane:~$ kubectl create -f prod-pod.yaml
pod/prod-pod created
vagrant@controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
prod-pod	1/1	Running	0	8s	10.244.140.103	node02	<none>		<none>	

- Created dev pod and it was scheduled in node01

```
vagrant@controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
dev-pod	1/1	Running	0	36s	10.244.196.162	node01	<none>		<none>	

➤ nodeAffinity

- The *nodeSelector* was very straight forward. You need to give *label* properly
[hard selection just like *matchLabels* in ReplicaSet]
nodeAffinity is having multiple types of label mathing. [like *matchExpressions* in ReplicaSet]

- What if you want to say like
place the pod *where env=prod or env=test*
place the pod *where env != dev*
in this case, ***nodeAffinity*** can be used.

- The yaml looks like the below

```
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In
                values:
                  - Large
```

```
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 5
          preference:
            matchExpressions:
              - key: zone
                operator: In
                values: [us-east-1]
```

- 2 types of nodeAffinity
 - required**DuringSchedulingIgnoredDuringExecution
 - preferred**DuringSchedulingIgnoredDuringExecution
- Pods are having 2 stage: *scheduling, execution*
 - ⌘ When the pod is not created in any node, it is first scheduled. [**Scheduling**]
 - ⌘ After the pod being scheduled, it'll be created in the scheduled node. [**Execution**]
 - ⌘ Currently, nodeAffinity is only there for *scheduling* phase, you can see its written Ignoredduring*Execution*.
- *required* one contains **nodeSelectorTerms** whereas *preferred* one contains *weight* and *preference* .
- In case of requiredDuringSchedulingIgnoredDuringExecution, if the expression doesn't match with any node, then the Pod will not be scheduled at all. [**HARD** matching]

In case of preferredDuringSchedulingIgnoredDuringExecution, all the nodes will be checked and weight will be calculated. Highest preferred node will be scheduled for the pod. [**SOFT** matching]
- How the *weight* is calculated in case of preferredDuringSchedulingIgnoredDuringExecution ?
 - ⌘ If you have provided some preferences, all will be checked in all the nodes available.

Each preference will be having some weight (you'll define this)
 - ⌘ The nodes will be checked for all the preferences. If preference matches in that node then weight for that node will be increased with the weight provided for that preference.

```
preferredDuringSchedulingIgnoredDuringExecution:
- weight: 5
  preference:
    matchExpressions:
      - key: env
        operator: In
        values: [prod]

- weight: 3
  preference:
    matchExpressions:
      - key: disk
        operator: In
        values: [ssd]
```

Node	env=prod	disk=ssd	Score
node1	✓	✓	5 + 3 = 8
node2	✓	✗	5 + 0 = 5
node3	✗	✓	0 + 3 = 3

here, *node1* has both the preference matched so its weight will be 8, so pods will be scheduled to *node1*.

- Use **kubectl explain pod.spec.affinity.nodeAffinity --recursive** to get the fields and directly paste inside pod yaml file.
- Here we used *matchExpressions* to match the label of the nodes, there is another called *matchFields*.
 - ⌘ It matches any type of fields you want, not just *labels*.

```
matchFields:
- key: metadata.name
  operator: In
  values: [node1]
```

[metadata.name must be node1]

- There are other 2 types of affinity, **podAffinity**, **podAntiAffinity**
 - ⌘ *nodeAffinity* checks the labels in the nodes and match that.
 - ⌘ *podAffinity* and *podAntiAffinity* checks the labels of the pods which are running inside the node.
- **podAffinity**

- ⌘ The pod running on target nodes will be checked with the preferences/ nodeSelectorTerms.
if matches then that node will be scheduled.
- podAntiAffinity
 - ⌘ Opposite of *podAffinity*,
the lowest matched (in case of *preferredDuring....*) will be selected.
or not matched (in case of *requiredDuring...*) will be selected.
- Some operators of *matchExpressions*

```
- matchExpressions:
  - key: size
    operator: In
    values:
      - Large
```

[In]

```
- matchExpressions:
  - key: size
    operator: Exists
```

[Exists]

 - ⌘ All the operators are
[kubectrl explain pod.spec.affinity.nodeAffinity.preferredDuringSchedulingIgnoredDuringExecution.preference.matchExpressions](#)

```
Possible enum values:
- ~"DoesNotExist"~
- ~"Exists"~
- ~"Gt"~
- ~"In"~
- ~"Lt"~
- ~"NotIn"~
```
- F

--- Resource Requirements ---

- There are 2 things, **Request** and **Limit**.
 - ⌘ Request: minimum required resource (used for scheduling)
 - ⌘ Limit: maximum allowed resource (enforced at runtime)
- The YAML look like this.

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"
```

[**resources** is written *inside containers*, not **spec**] < -----

- **requests** (Scheduling Decision)
 - ⌘ If any node has at least this much free resources, then only that node can be scheduled for the pod.
Otherwise it'll not be scheduled.
- **limits** (Runtime enforcement)
 - ⌘ CPU: throttled (if exceed than the limit set) [throttled means slow down]
 - ⌘ Memory: container is *killed* (**OOMKilled**) (if exceed than the limit set)
- CPU vs Memory behaviour
 - ⌘ CPU can exceed request, but Memory cannot.
 - ⌘ In case of hitting the limit,
CPU: throttled [slow down]
Memory: *container killed*

- Resource is not there, then by default it takes as 0.
But, if resource is not there and limit is there: resource will be taken same as limit.
If limit is not there, then it'll assume the pod can use *unlimited* resources.
- 1 (one) count of CPU is same as
 - ⌘ 1 AWS vCPU
 - ⌘ 1 GCP Core
 - ⌘ 1 Azure Core
 - ⌘ 1 Hyperthread
- Memory
 - ⌘ 1 G : 1 Gigabyte
 - ⌘ 1 M : 1 Megabyte
 - ⌘ 1 K : 1 Kilobyte
 - ⌘ 1 Gi : 1 Gibibyte
 - ⌘ 1 Mi : 1 Mebibyte
 - ⌘ 1 Ki : 1 Kibibyte

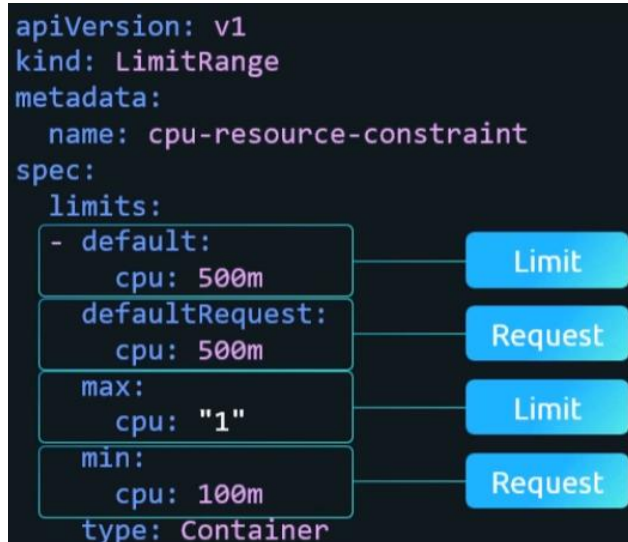
[K : 1000 bytes, Ki: 1024 bytes ; K is binary (2's power), Ki is decimal (10's power)]
- We can have 4 scenarios:
 - No requests, No limits
 - No requests, Limits
 - Requests, Limits
 - Requests, No limits
 - ⌘ In the first case, if one pod (container inside pod because requests and limits are set per container, not per pod) is consuming all the CPU, and another pod is also assigned to same node (because if request is not given, it'll be **taken as 0**) which is not able to get the resources.
 - ⌘ In the second case, request will be **taken as same as limits**.
 - ⌘ In the 3rd case, the pods are having both request and limits. Lets say in this case pod-1 doesn't need extra resources but pod-2 needs.
And we have set the limit for both the pods which will stop *pod-2* to use more resources. Which is not ideal
 - ⌘ In the 4th case, the problem of 3rd case can be solved. Because if a pod is being assigned to a node means the node has required amount of resources

available that the pod needs.

And, there is no limit so pod can use as much as it want.

- There is something called **LimitRange**, it by default injects the *requests* and *limits* to the pods if not specified in the yaml.

♣ It works in **namespace level**.



♣ **default :**

- ♣ it is the *limits* field.
- ♣ If the pod yaml doesn't contain *limits* fields, then this LimitRange's *spec.limits.default* will be added there.

♣ **defaultRequest :**

- ♣ It is the *requests* field.
- ♣ If the pod yaml doesn't contain *requests* fields, then this LimitRange's *spec.requests* default will be added there.

♣ **min**

- ♣ The minimum amount of resource a container can ask for (*requests*).
- ♣ If the container (inside pod) asks lesser resource then this *min* then it'll be rejected.

♣ **max :**

- ♣ Same as *min*.
- ♣ The container cannot request more requests then that defined in *max*.

♣ **type : Container**

➤

➤ DaemonSet

- ⌘ It is similar to *ReplicaSet*
- ⌘ It makes sure that each node should contain one copy of the pod specified in *DaemonSet* configuration.
- ⌘ Earlier it used to append *nodeName* field and POST request to create the container in the target node.
But now, it uses *nodeAffinity* to assign a specific node to the Pod.
- ⌘ The yaml is exact same as that of *ReplicaSet*, just the *kind* will be different.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
```

⌘
➤ F

- Static Pods -

- Pods are created by the command of *kube-api-server*, which is listened by *kubelet* (in worker nodes) and create and start the container.
what if the worker node is not connected to any Kubernetes cluster, it just has *kubelet* installed. How to run a pod here?
- You need to find the *staticPodPath* in the node where *kubelet* is present.
kubelet constantly see this path and create and update the pod if any *yaml* config is present inside this path.
- **NOTE :-** You can only create pods in this way, other resources like *ReplicaSet*, *Deployment* cannot be created because they are completely of *Kubernetes* control plane things.
- To find the *staticPodPath* the below methods are there [all in worker node, not master node]

♣ Method-1

♣ `systemctl cat kubelet`

```
# Note: This dropin only works with kubeadm and kubelet v1.11+
[Service]
Environment="KUBELET_KUBECONFIG_ARGS=--bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf"
Environment="KUBELET_CONFIG_ARGS=--config=/var/lib/kubelet/config.yaml"
# This is a file that "kubeadm init" and "kubeadm join" generates at runtime, populating the KUBELET_KUBEADM_ARGS variable
EnvironmentFile=/var/lib/kubelet/kubeadm-flags.env
# This is a file that the user can use for overrides of the kubelet args as a last resort. Preferably, the user should
# the .NodeRegistration.KubeletExtraArgs object in the configuration files instead. KUBELET_EXTRA_ARGS should be
EnvironmentFile=/etc/default/kubelet
ExecStart=
ExecStart=/usr/bin/kubelet $KUBELET_KUBECONFIG_ARGS $KUBELET_CONFIG_ARGS $KUBELET_KUBEADM_ARGS $KUBELET_EXTRA_ARGS
```

(you'll get something like this; this image is cropped)

if you find something like below, then see the `--pod-manifest-path`

```
ExecStart=/usr/local/bin/kubelet \
--container-runtime=remote \
--container-runtime-endpoint=unix:///var/run/containerd.sock \
--pod-manifest-path=/etc/kubernetes/manifests \
--kubeconfig=/var/lib/kubelet/kubeconfig \
--network-plugin=cni \
--register-node=true \
--v=2
```

- ♣ Open that *config.yaml* file present there, you'll find a key called *staticPodPath*

```
staticPodPath: /etc/kubernetes/manifests
```

- ♣ In this path i.e. */etc/kubernetes/manifests*, if you create a Pod's *yaml* file, then *kubelet* will create that pod inside that node.

- The above displayed is `kubelet.service` file (it is same as the above `systemctl cat kubelet`)
- Its mostly in `/usr/lib/systemd/system/kubelet.service`
if not, then find other service file directories like
`/etc/systemd/system`, `/var/lib/systemd/system` ..etc
- There you'll find the `ExecStart` field

```
[Unit]
Description=kubelet: The Kubernetes Node Agent
Documentation=https://kubernetes.io/docs/
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/usr/bin/kubelet
Restart=always
StartLimitInterval=0
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Method-2

- `ps -ef | grep kubelet`

```
vagrant@node01: /usr/local/bin$ ps -ef | grep kubelet
root      3762      1   2 May02 ?           00:32:17 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf --config=/var/lib/kubelet/config.yaml --node-ip 192.168.56.21
root      4046      1   2 May02 ?           00:00:05 /usr/bin/node-driver-registry --v=5 --csi-address=/
```

- You'll find the `config.yaml` path here.
- Go to that `yaml` file and get the `staticPodPath`.

➤ To check in the worker node

- You can use `crictl` command to see if the container is running.
You can't use `kubectl` command there because its not kubernetes.
- `crictl` is used to talk to `containerd`.

If you get "Permission Denied" then run with `sudo`.

- `sudo crictl ps`

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD	NAMESPACE
306e8944e30c3	6c3a6ea6608c8	26 minutes ago	Running	static-cont	0	37b86e808bd54	test-static-pod-node01	default
007131327410e0	6c3a6ea6608c8	16 hours ago	Running	dev-cont	0	65d3e3016d3e0	dev-pod	default

- kubeadm creates **static pods** in master node to run the **control plane resources**.

```
vagrant@controlplane: /etc/kubernetes/manifests$ ls
etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml
```

- When you execute `kubectl get pods` in the control plane, you'll see the **static pod** there.

HOW? Because this should be specific to the worker node and kube API

Server shouldn't know about this. Read about the *end-to-end flow* (next page) for this.

| End-To-End flow from Pod creation command to actual Running of

Pod |

----- IMPORTANT -----

- Below is the flow end-to-end
 - ⌘ User creates a Pod
 - ✦ `kubectrl apply -f my-pod.yaml` (or) `kubectrl run ...`
 - ✦ This request goes to API Server.
 - ⌘ API Server
 - ✦ It validates the request.
 - ✦ Store the *Pod* object (PodSpec) in *etcd*.
 - ⌘ Scheduler
 - ✦ It sees the unscheduled pod in *etcd*.
 - ✦ It schedule any node to this pod [updates *spec.nodeName*]
 - ⌘ Kubelet (in the scheduled node)
 - ✦ Watches API Server, and see a pod is assigned to it.
[kubelet opens a *watch* connection. API Server streams updates through it]
 - ✦ Pulls the *PodSpec*.
 - ⌘ Kubelet → Container runtime (*containerd*)
 - ✦ Creates Pod sandbox (pause container: otherwise container will get its own IP; instead of using Pod's IP)
 - ✦ Creates all the required containers defined in *PodSpec*.
 - ⌘ Container Runtime (containerd)
 - ✦ Pulls images, & creates the containers.
 - ✦ Assign *container IDs*.
 - ✦ Start the processes.
 - ⌘ Kubelet (after creation)
 - ✦ Maps *Pod ID* → *Container IDs*
 - ✦ Monitors the containers
running / crashed / restarted
 - ⌘ Kubelet → API Server (status update)
 - ✦ API server doesn't poll for Pod's status.
Kubelet update API server about the Pod's status.
 - ✦ API Server stores updated status in *etcd*.


```

status:
  phase: Running
  containerStatuses:
    - name: nginx
      containerID: containerd://abc123
      state: running

```

- ⌘ Control Plane (now updated)
 - ⌘ *etcd* contains
 - PodSpec (desired state)
 - PodStatus (actual state from Kubelet)
- ⌘ Controllers (ReplicaSets, etc..)
 - ⌘ Watch API Server
 - ⌘ Compares
 - Desired state vs Current state
 - ⌘ Take action if mismatch.
- Now, the question arises: Does the ReplicaSet or other controllers repeatedly poll API Server about the pod's status?
 - ⌘ No, it starts a *watch event*.
 - ⌘ Its not a *web-hook*, it's a long-lived connection (*streaming connection*)
 - ⌘ Whenever something changes, API Server streams it, so *controllers* get to know about the pod's status.
- Control Plane knows nothing about the containers.

It gets the details about the containers (id and status) via Kubectl only.

Component	Knows PodSpec	Knows Container ID	Runs Containers
API Server	✓	⚠ (reported only)	✗
etcd	✓	⚠ (stored only)	✗
kubelet	✓	✓ (authoritative)	✗
containerd	✗	✓ (real)	✓

- To see the details about the containers in *Control Plane* and *Worker Node*
 - ⌘ When you execute


```
kubectl get pod <pod name> -o yaml
```

(or)

```
kubectl edit pod <pod name>
```

 - ⌘ It shows the content of *etcd* in yaml format.

⚡ **status.containerStatuses**

this field contains the list of containers associated with that pod along with their IDs, image, state (running, etc.), volumes ..ec.

```
containerStatuses:
- containerID: containerd://98243133419e0d62
  image: docker.io/library/nginx:1.29
  imageID: docker.io/library/nginx@sha256:6e
  lastState: {}
  name: dev-cont
  ready: true
  resources: {}
  restartCount: 0
  started: true
  state:
    running:
      startedAt: "2026-05-02T18:10:45Z"
  user:
    linux:
      gid: 0
      supplementalGroups:
        - 0
      uid: 0
  volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.:
```

➤ **NOTE**

- ⚡ *kubelet* has a watch connection with API Server.
And, whenever API Server gets some request (either create a new pod or update an existing pod), it updates the *etcd*.
- ⚡ When you trigger *kubectl edit pod <pod name>*
 - ⚡ API Server gets the update request, it updates the *etcd* if there is any changes.
 - ⚡ *kubelet* already has a *watch connection*, through that API Server sends the modified *PodSpec*.
 - ⚡ *kubelet* gets the new *PodSpec* and compare with the existing one.
 - ⚡ If any changes, then it recreates the container (if allowed; if not allowed then reject it)
it sends the report of new container to the API Server i.e. new *Container ID*, ..etc.

```

containerStatuses:
- containerID: containerd://3d835f86cfac50fe61ea18455f9b7a9a22f98e73aa22d2c34dd54d4c7bf9b76b
  image: docker.io/library/nginx:1.29-perl
  imageID: docker.io/library/nginx@sha256:3d5526f7dc2dfd0b6cbebc664ca2d7ab1e1b5f572a289c6a7607728c5f5b85c
  lastState:
    terminated:
      containerID: containerd://98243133419e0d62baa1134c86c58fa102f93170f3ff7f82e08ebce518078c96
      exitCode: 0
      finishedAt: "2026-05-03T13:08:32Z"
      reason: Completed
      startedAt: "2026-05-02T18:10:45Z"

```

[you can see, *lastState.terminated*; it contains previous containerID here]

[contains only last terminated container details, not all]

- In case of no change, doesn't do anything.
- In case of **static pod**, kubelet sends the same report to API Server; but **with some extra fields**
- The below is the *metadata* of normal pod (pod created by api server)

```

metadata:
  annotations:
    cni.projectcalico.org/containerID: c6d2a3016d3a055cededa48f1ca2c97a5c69303ce534161abd71d26a4e4c2641
    cni.projectcalico.org/podIP: 10.244.196.162/32
    cni.projectcalico.org/podIPs: 10.244.196.162/32

```

- The below is the *metadata* of **static pod** (pod created in worker node itself)

```

metadata:
  annotations:
    cni.projectcalico.org/containerID: 37b86e808bd5439f78bf763fa7914144be4b67de4342957d3253bc2412eaa40a
    cni.projectcalico.org/podIP: 10.244.196.163/32
    cni.projectcalico.org/podIPs: 10.244.196.163/32
    kubernetes.io/config.hash: af6e265c669e39311b5f3774274e0832
    kubernetes.io/config.mirror: af6e265c669e39311b5f3774274e0832
    kubernetes.io/config.seen: "2026-05-03T09:51:34.943426976Z"
    kubernetes.io/config.source: file

```

- You can see 2 key value pairs:
kubernetes.io/config.mirror: <hash>
kubernetes.io/config.source: file
 these tells API Server that **this is not a Pod created by API Server; this is a mirror of a static pod.**
- **config.source: file** means the pod is created from a **file** present in the **node** itself;
 if this field is not there means the **pod** is created by **API Server** itself.

- The API server stores mirror pod objects also in etcd, but they are not the source of truth. They are copies of static pods created by kubelet and marked with annotations to indicate their origin.



Resources inside Pods vs Resources as normal Processes

- As we saw the control plane processes run in *static pods* in the control plane node when you configure the cluster via *kubeadm*, how the things communicate with each other then?

And, how the resources communicates with each other when they are not running inside any *pod* rather running outside normally as a process?

- First lets see, how they work when run as normal processes.

- ⌘ There are multiple *configuration* files inside */etc/kubernetes* directory.

```
vagrant@controlplane:/etc/kubernetes$ ls
admin.conf  controller-manager.conf  kubelet.conf  manifests  pki  scheduler.conf  super-admin.conf
```

- ⌘ All these files contains the specific requirements of each processes like *scheduler, controller manager ...etc.*
- ⌘ All the *.conf* files are having same structure as the *~/.kube/config* file, but **user, certificates, permissions** are different.
- ⌘ In the *scheduler.conf* the user is **system:kube-scheduler**

```
users:
- name: system:kube-scheduler
  user:
```

- ⌘ *kubelet, kubect, controller, scheduler* all talks to API Server.
API Server only directly talks to *etcd*.

```
scheduler —> API Server —> etcd
controller —> API Server
kubelet —> API Server
kubectl —> API Server
```

[No component talks to *etcd* directly except *API Server*]

- ⌘ For example, if you see the *config* files, the *server* will be having **<control plane node's IP>:6443**
because, API Server listens on port 6443.

```
vagrant@controlplane:/etc/kubernetes$ sudo cat admin.conf | grep server
server: https://192.168.56.11:6443
vagrant@controlplane:/etc/kubernetes$ sudo cat scheduler.conf | grep server
server: https://192.168.56.11:6443
vagrant@controlplane:/etc/kubernetes$ sudo cat kubelet.conf | grep server
server: https://192.168.56.11:6443
```

- ⌘ All the server contains this end-point to talk to [because all talk to *API Server* only]
And, all the communication happens **via HTTP Calls**.

⌘ This is how the communication and work happens in *control plane* nodes.

➤ **Now, how the resources behaves when all are present inside Pods**

⌘ As we saw, all the resources communication happens via *HTTP Calls* only.
So, even if they are present inside the pods, there will some port mapping like

6443:6443 (just example)

⌘ So, nothing changes. All the end-points remains same.

⌘ *kubeadm* keeps the same port mapping from host to pods, so the end-points doesn't change.

In case if you want to bind different host port to the pod, then *configuration files* needs to be updated with this.

And, all of the things will work now.

⌘ *kubeadm* runs all the resources in *static pods* (not normal pods).

So, *API Server* or *etcd* has nothing to do with this.

[[[as we know, even if the pods are static, when you execute *kubectrl get pods* command you'll be able to see the static pods because *kubelet* informs *API Server* about the static servers as well and *API Server* stores the details about the *static pods* inside *etcd*; but the *static pods* are having 2 extra fields inside *annotations* which are *config.mirror: <hash>* and *config.source: file*]]]
when you see the details about the *resources pods*, you'll find the fields inside *annotations*.

```
metadata:
  annotations:
    kubernetes.io/config.hash: caf1ce5bf82be95b0c98349010a12fde
    kubernetes.io/config.mirror: caf1ce5bf82be95b0c98349010a12fde
    kubernetes.io/config.seen: "2026-04-30T15:07:40.894345870Z"
    kubernetes.io/config.source: file
```

(kube-scheduler pod)

⌘ So, *API Server* cannot update these pods because *static pod* can only be updated if you change the *yaml* files directly inside *manifest* (or any directory configured) directory.

⌘ The main advantage of running the resources inside *static pods* is if the pods crashes or the resources fails, *kubelet* will **automatically restarts** the pods by itself.

➤ **NOTE :::** *kubelet* doesn't run inside any pod, it is a *systemd* service;

So, when you execute *systemctl cat kubelet* you get the *kubelet.service* file content.

But, it'll not work for kube-apiserver, etcd, kube-scheduler ..etc because they run inside Pods.

----- Multiple Schedulers -----

- There are 3 different files, that might be confusing but they are different.

- ♣ **kube-scheduler.yaml**

present inside */etc/kubernetes/manifest* (Static Pod's yaml)

- ♣ **scheduler.conf**

kube-scheduler config file (same structure as of *~/.kube/config* file)

- ♣ **scheduler-config.yaml**

[not same as *scheduler.conf*]

it is the yaml file to create **KubeSchedulerConfiguration** resource.

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
```

- ♣ In simple words:

kube-scheduler.yaml creates the pod to run scheduler within it.

scheduler.conf is the configuration file (details and certificates) for that scheduler.

scheduler-config.yaml is the extra configuration plugins etc.

- When you open the *kube-scheduler.yaml* (kind: Pod, where kube-scheduler runs), you'll see a **command** field (which is run inside the container).

```
spec:
  containers:
  - command:
    - kube-scheduler
    - --authentication-kubeconfig=/etc/kubernetes/scheduler.conf
    - --authorization-kubeconfig=/etc/kubernetes/scheduler.conf
    - --bind-address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
    image: registry.k8s.io/kube-scheduler:v1.36.0
```

[**kube-scheduler** commands comes with that *image* mentioned here]

- ♣ Here, the *scheduler-config.yaml* is not mentioned, rather directly all the things are written here.

- ⌘ In modern way, *scheduler-config.yaml* is present which is more proper.

```
spec:
  containers:
    - name: kube-scheduler
      image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
      command:
        - kube-scheduler
        - --address=127.0.0.1
        - --kubeconfig=/etc/kubernetes/scheduler.conf
        - --config=/etc/kubernetes/my-scheduler-config.yaml
```

[the name would be *scheduler-config.yaml* in case of default one]

- After creating your custom scheduler, you need to mention this explicitly inside Pod's yaml file to choose the *custom scheduler* instead of the default one.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
  schedulerName: my-custom-scheduler
```

- F

Scheduler Profiles

- Till now we saw the below things:
 - ⌘ One *yaml* file is there to create the Pod containing *scheduler* process (running scheduler binaries) [*/etc/kubernetes/manifest/kube-scheduler.yaml*]
 - ⌘ One *config* file is there which contains the details about *cluster*, *user*, *context* to authenticate and authorize kube-scheduler.
[*/etc/kubernetes/scheduler.conf*]
 - ⌘ One *yaml* file is there which contains the *configurations* of the *scheduler*
[*scheduler-config.yaml*]

```
# my-scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
```

[for example]

it's reference will be there in the *command* section of *kube-scheduler.yaml* (in modern systems) or it'll not be present (in older systems the configs are directly given in that command section)

- The Pod scheduling process comprises of mainly below phases:
 - ⌘ **Scheduling queue**
all the unscheduled pods will be kept in a queue according to their priority.
 - ⌘ **Filter phase**
scheduler eliminates the nodes which do not satisfy the requirement of the Pod's resource requirement.
 - ⌘ **Scoring phase**
nodes that pass *filter phase* are now scored. The node that is scored more will be scheduler.
 - ⌘ **Binding phase**
the node who got scored more score will be binded to the pod.

- There are different *plugins* being used during this scheduling process.



⌘ *PrioritySort* plugin

during the scheduling queue phase, this plugin orders the Pods based on their priorities.

⌘ *NodeResourceFit* plugin

it is used during *filter phase* to exclude nodes.

Also, during *scoring phase* it score the nodes that pass in *filter phase*.

⌘ *NodeUnschedulable* plugin

it makes sure that the nodes marked as *Unschedulable: true* will not get the pod assigned.

```
vagrant@controlplane: /etc/kubernetes$ kubectl describe node node01
Name: node01
Roles: <none>
Labels: beta.kubernetes.io/arch=amd64
        beta.kubernetes.io/os=linux
        env=dev
        kubernetes.io/arch=amd64
        kubernetes.io/hostname=node01
        kubernetes.io/os=linux
Annotations: csi.volume.kubernetes.io/nodeid: {"csi.tigera.io": "node01"}
              node.alpha.kubernetes.io/ttl: 0
              projectcalico.org/IPv4Address: 192.168.56.21/24
              projectcalico.org/IPv4VXLANTunnelAddr: 10.244.1.1
              volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp: Thu, 30 Apr 2026 15:26:20 +0000
Taints: <none>
Unschedulable: false
```

⌘ *ImageLocality* plugin

this just gives a *soft preference* if the node already contains the *required container image*.

⌘ *DefaultBinder plugin*

in the binding phase, this plugin finalizes the Pod-to-Node assignment.

- In the above image, each stage is further divided into sub-stages like
queueSort

preFilter, filter, postFilter

preScore, score, reserve

permit, preBind, bind, postBind

- ⌘ All these are called **extension points**.

- ⌘ With the *extension points*, the **plugins** are binded.

- We can create multiple **scheduler pods** and run as many schedulers as we want.
And for each scheduler, we can create a **scheduler configuration** file (yaml)
(**kind: KubeSchedulerConfiguration**).

- ⌘ In this *KubeSchedulerConfiguration*, we can choose **which plugins we want to enable/disable for the extension points**.

- ⌘ Let's say you have 2 pods to be scheduled, and you want different types of *plugins* will be used in different *extension points* in those.

Then, you can create 2 *scheduler pods* and write 2 *kube scheduler configuration* each having specific plugins enabled/disabled.

```
# my-scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
```

```
# my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler-2
```

- ⌘ But instead of doing this, we can just create one scheduler and attach multiple profiles to it.

[*scheduler* is the **process** that is running inside the Pod.

Profiles are the **configurations having specific modules enabled/disabled at the extension points**]

So, instead of creating multiple scheduling processes (pods), just create one scheduling process and attach multiple profiles to it.

```

apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler-2
  plugins:
    score:
      disabled:
        - name: TaintToleration
      enabled:
        - name: MyCustomPluginA
        - name: MyCustomPluginB

- schedulerName: my-scheduler-3
  plugins:
    preScore:
      disabled:
        - name: "*"
    score:
      disabled:
        - name: "*"

```

[2 scheduler profiles are

there]

- ⌘ The question arises, if we have multiple pods then we'll have concurrency if the pods chooses different profiles, but in this case there will not be having concurrency.

ALL THE PROFILES ATTACHED TO A SCHEDULER RUNS IN
MULTIPLE THREADS.

- ⌘ The nesting are like the below:

plugins >> **extension point** >> **disabled/enabled** >> *list of name*
(of plugins)

➤ Leader Selection

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
  leaderElection:
    leaderElect: true
    resourceNamespace: kube-system
    resourceName: lock-o
```

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
  leaderElection:
    leaderElect: false
```

[have to check if

leaderElection is scheduler level or profile level]

- ⌘ If you enable the leader selection in all schedulers, **only one scheduler** will be active at a time.
- ⌘ If you disable the leader selection then **all the schedulers** can be active at same time.
- ⌘ Lets say you have 2 schedulers and 4 profiles
s1: p1 p2
s2: p3 p4
and all the profiles are different (i.e. the *schedulerName* are different)
 - ⌘ **leaderElect: true**
in this case lets say your pod wants p4 (scheduler profile) and the leader got selected is *s1*.
as *s1* doesn't contain the required profile i.e. *p4* the pod will remain pending because the selected leader can't schedule the pod.
 - ⌘ **leaderElect: false**
in this case, both *s1* and *s2* will be active and in this case *s2* (*p4*) will schedule the pod to specific node.
- ⌘ If you keep *leaderElect: false*, its obvious race condition will occur (if duplicate *schedulerName* are present across the schedulers),

even in that case, **multiple scheduling doesn't happen in kubernetes**. If one scheduler schedule the node, then another scheduler will not be able to do.

➤ How to read from the *docs* ?

- ⌘ For all the configuration related things like *KubeSchedulerConfiguration*, ..etc etc, you can directly search in the docs.

kube-scheduler Configuration (v1) - Kubernetes

Kubernetes › docs › reference › config-api › kube-scheduler-config

[you'll get links like this]

- ⌘ For the *Configuration* (you'll NOT get the docs usnig *kubectl explain* command because these are not resources, these are just the configurations), you should go to that *reference > config-api* path only (most of the cases).
- ⌘ Inside that you'll see a lot of things like this.

ClientConnectionConfiguration [↗](#)

Appears in:

- [KubeSchedulerConfiguration](#)

ClientConnectionConfiguration contains details for constructing a client.

[you can see here written: *Appears in: KubeSchedulerConfiguration*]

- ⌘ Then when you click that link (*KubeSchedulerConfiguration*)

leaderElection [Required]

[LeaderElectionConfiguration](#)

clientConnection [Required]

[ClientConnectionConfiguration](#)

DebuggingConfiguration [Required]

[DebuggingConfiguration](#)

you'll see some fields like this.

- ⌘ It means, the previous field [*ClientConnectionConfiguration*] values can be written under *clientConnection* field inside *KubeSchedulerConfiguration* .

⌘

Security Primitives

- RBAC: Roll Based Access Control
 - ABAC: Attribute Based Access Control
 - Node Authorization
 - Webhook Mode
- How to secure the cluster?
 - ⌘ The core component is API Server. It is the single point of contact for all operations, whether it is via *kubectl* or through *API*
 - ⌘ Who can access the cluster?
 - ⌘ Which actions can they perform once access is granted?

Authentication

- In a normal web application backend (what we usually build), the username and password are stored in database and using *JWT* token we verify the user. But, Kubernetes doesn't have any database to store the user details. So it depends on an external provider like *static files*, *certificate authorities*, *third party identity provider like LDAP*. ---- for authentication **human users**.
[Application level user security is managed within the application itself]
- **Static file**

- ⌘ You can create a *csv* file and write the user details there.

```
password123,user1,u0001,group1
password123,user2,u0002,group2
password123,user3,u0003,group2
```

If you want *tokens*, you can write the same (write token in place of password column)

```
KpjCvBI7rCFAHYPKByTlzRb7guLcUc4B,user10,u0010,group1
rJjncHmvtXHc6M1WQddhtvNyhgTdxSC,user11,u0011,group1
mjp0FEiFokLgtoikaRNTt59ePtczZSq,user12,u0012,group2
```

[4th column (group) is completely **optional**]

- ⌘ Then you can mention this file inside the config.
If kube-apiserver is running as a *systemd* service (not running inside Pod)

```
ExecStart=/usr/local/bin/kube-apiserver \
  --authorization-mode=Node,RBAC \
  --advertise-address=... \
  --basic-auth-file=user-details.csv
```

if *kube-apiserver* is running inside *Pod* then write inside the configuration *yaml* file.

```
spec:
  containers:
  - name: kube-apiserver
    image: k8s.gcr.io/kube-apiserver-amd64:v1.11.3
    command:
      - kube-apiserver
      - --authorization-mode=Node,RBAC
      - --advertise-address=172.17.0.107
      - --basic-auth-file=/path/to/user-details.csv
```

- ♣ In case of *username password* the request will be like the below

```
curl -v -k https://master-node-ip:6443/api/v1/pods -u
"user1:password123"
```

in case of *username token* the request will be like below

```
curl -v -k https://master-node-ip:6443/api/v1/pods --header
"Authorization: Bearer df897ejkhefrt"
```



Generate Certificates for Cluster

➤ One small Analogy about CA ----- IMPORTANT -----



- ♣ Lets assume CA as Government Authority.
 - ✦ It first finds its own identity key [government stamp | *ca.key*]
 - ✦ then it creates its own registration form [*ca.csr*]
 - ✦ then, it sign that by itself using that stamp [*ca.crt*]
 - ✦ [because, the authority has to self sign its own document]
- ♣ Now, this government certificate has to be made the source of TRUST so that whatever signed by this will be also a source of TRUST.
- ♣ Then, one user came with its details to make it verified.
 - ✦ Admin user has its own signature [signature | *admin.key*]
 - ✦ it creates a application form like aadhar card details [*admin.csr*]

- ✦ Government authority has to approve it with their own stamp
[*admin.crt*]
[here, *ca.key* was used to sign *admin.csr* ; *ca.crt* didn't sign but it is the
TRUST that this certificate belongs to a trusted authority]



- There are multiple tools that provides certificates like OPENSLL, EASYRSA, CFSSL. We'll use **OpenSSL** here.
- Will start with **CA** (Certificate Authority) certificate [SELF SIGNED]

➤ All the components that needs Client or Server or Both certificates.

⌘ Components that acts as *ONLY* Client

[these all calls the API Server → needs *Client certificate*]

- ⌘ kubectl (admin users)
- ⌘ kube-scheduler
- ⌘ kube-controller-manager
- ⌘ kube-proxy

⌘ Components that acts as *BOTH* Client and Server

⌘ **kube-apiserver**

client cert → talks to etcd, kubelet

server cert → serves kubectl / components

⌘ **kubelet**

client cert → talks to API server

server cert → API server calls it (exec/logs)

⌘ **etcd**

server cert → serves API server

client cert → when it talks peer-to-peer (clustered etcd)

➤ For the clients to validate certificates sent by server and vice-versa, they all need a copy of *ca.crt* (certificate authority).

➤ Read this Docs to setup SSL for each server and client components.

<https://notes.kodecloud.com/docs/Kubernetes-and-Cloud-Native-Associate-KCNA/Container-Orchestration-Security/TLS-in-Kubernetes-Certificate-Creation/page#generating-server-side-certificates>



TLS in Kubernetes

- Generating CA

5

- ```
-----BEGIN PRIVATE KEY-----
MIIEVwIBADANBgkqhkiG9w0BAQEFAASCBKwwggSIAgEAAQIBAQCISIG1tgzfbtL91
YV+jFwzMpPd+Jf64u1kS5fBgrrL0Ce9qsDf7ISyRjqhJaUkiALJ1BUGQDeD5ceno
GjwSxphuE1817jQD2i+/tLAZ7Vnzo1602Ev1TC7YdaRZ6H1ZyrStij205wEh1+pD
```

- ```
-----BEGIN CERTIFICATE REQUEST-----
MIITCTCCAUAUQAQAwGDEWMBQGA1UEAwNS1VCRVRJORVRUy1DQTCASIAwDQYJKoZI
hvcNAQEBBQADggEPADCCAQoCggEBAKwgbw2DN9u0v3VhX6MXDMyk934l/r16Wtnl
8Eausv0172qWw/zhlJG0FlpSSTasmfE0ZANAP1y6ogaVZlErg4Wl/zXs4Pal+0t
```

- ```
-----BEGIN CERTIFICATE-----
MIICtzCCAZ8CFEdD/85/RjnbfaF0lyVomsIAIzeOMA0GCsqGSIB3DQEBCwUAMBgx
FjaUBGAmbAMMDUTvQkvSTkvrVRMTQ0EwhhcNMjYwNTA0MTk1MjI0WhcNMjYwNjAz
MTEtMTUyOTUyAYMDK6FAYPXQODPA11MU7FLHFEVEWELUNRMTBRTANPdkbhkiCo
```

- ## ➤ Creating Client Certificates

- Now we'll create certificates for  
*admin (kubectl), scheduler, controller manager, kube-proxy*

➤ Client Certificate: *Admin*

```
openssl genrsa -out admin.key 2048
```

```
openssl req -new -key admin.key \
-subj "/CN=kube-admin/O=system:masters" \
-out admin.csr
```

[ here, CN = username (or) identity (*kube-admin* in here)

O = RBAC group (*system:masters* in here) ]

- system:masters**

built-in super user group having full access to the cluster, and bypass RBAC restrictions.

**kubectl get clusterrolebindings cluster-admin -o yaml**

```
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
 kind: Group
 name: system:masters
```

[ **system:masters** group is cluster

admin ]

- Sign the certificate with CA key and certificate.

```
openssl x509 -req -in admin.csr \
-CA ca.crt -CAkey ca.key \
-out admin.crt
```

- Instead of *username + password*  
*certificate + private key* [ it'll be used ]

➤ Client Certificate: OTHERS

- Same process as *admin*, just the CN will be changed for others.
- They'll create *key, certificate request* and  
CA will sign those to generate *certificate*.

➤ Using Client Certificate

- Steps:
  - Client sends request
  - Server asks for certificate
  - Client sends admin.crt
  - Server:



- \* verifies CA
- \* extracts CN/O
- Applies RBAC
- To use the certificates to talk to *kube-apiserver*, you have 2 options.  
*kubectl, curl* (with key and certificates)

```
apiVersion: v1
clusters:
- cluster:
 certificate-authority: ca.crt
 server: https://kube-apiserver:6443
 name: kubernetes
kind: Config
users:
- name: kubernetes-admin
 user:
 client-certificate: admin.crt
 client-key: admin.key
```

[ add in the *config* file and use

*kubectl* to talk to switching to the required *context* ]

```
curl https://kube-apiserver:6443/api/v1/pods \
--key admin.key --cert admin.crt \
--cacert ca.crt
```

[ or you can

directly pass the *admin key and certificates, CA certificate with curl* ]

- Creating Server Certificates
- Server Certificate: *etcd*

```
openssl genrsa -out etcd-server.key 2048
```

```
openssl req -new -key etcd-server.key \
-subj "/CN=etcd" \
-out etcd-server.csr
```

```
openssl x509 -req -in etcd-server.csr \
-CA ca.crt -CAkey ca.key \
-out etcd-server.crt
```

- As *etcd* is a distributed database across the cluster, so it needs to talk to the *peer etcd* as well.

One *etcdserver.crt, etcdserver.key* will be there. And remaining will be for *etcdpeer* i.e.

*etcdpeer1.crt, etcdpeer1.key*

*etcdpeer2.crt, etcdpeer2.key*

.... etc

```
- etcd
- --advertise-client-urls=https://127.0.0.1:2379
- --key-file=/path-to-certs/etcdserver.key
- --cert-file=/path-to-certs/etcdserver.crt
- --client-cert-auth=true
- --data-dir=/var/lib/etcd
- --initial-advertise-peer-urls=https://127.0.0.1:2380
- --initial-cluster=master=https://127.0.0.1:2380
- --listen-client-urls=https://127.0.0.1:2379
- --listen-peer-urls=https://127.0.0.1:2380
- --name=master
- --peer-cert-file=/path-to-certs/etcdpeer1.crt
- --peer-client-cert-auth=true
- --peer-key-file=/etc/kubernetes/pki/etcd/peer.key
- --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
- --snapshot-count=10000
- --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

[ it

takes the keys and certs like this (*server key & crt, peer key & crt, etcd CA crt*)  
in the *etcd* yaml file ]

**NOTE:** Kubernetes CA is a root, and etcd CA is a root. They don't trust each other.

➤ Server Certificate: *kube-apiserver*

- ⌘ It's the most important thing because it is the core of Kubernetes cluster that is being used to talk to the whole cluster.

Either via *curl* (direct http REST call) or via *kubectl*, all the request comes to *kube-apiserver* only and it will be delegated from this only.

- ⌘ *kube-apiserver* is both *server* and *client* component.

Client: it talks to *kubelet*, *etcd*

Server: serves *kubectl* and other components.

- ⌘ It uses 2 different CA for its different certificates.

When it acts as a server (for *kubectl*, *kubelet*, ..etc), it gets certificate by Kubernetes CA. [ *apiserver.crt* ]

When it acts as a client (talking to *etcd* ), it gets certificate by etcd CA  
[ *apiserver-etcd-client.crt* ]

```
--tls-cert-file=/etc/kubernetes/pki/apiserver.crt
--tls-private-key-file=/etc/kubernetes/pki/apiserver.key
--client-ca-file=/etc/kubernetes/pki/ca.crt
```

```
--etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
--etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
--etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
```

Now the question is, how Kubernetes trust 2 different CAs at once?

Kubernetes doesn't have a single global trust store.

Each connection explicitly defines which CA to trust.

Trust is not global,

Trust is per connection.

```
Connection 1 (incoming clients):
 → use Kubernetes CA

Connection 2 (to etcd):
 → use etcd CA
```

You can see the certificates (client and server) and their respective CAs

```
vagrant@controlplane:/etc/kubernetes/pki$ ls
apiserver-etcd-client.crt apiserver-kubelet-client.crt apiserver.crt ca.crt etcd
apiserver-etcd-client.key apiserver-kubelet-client.key apiserver.key ca.key front-proxy-ca.crt
vagrant@controlplane:/etc/kubernetes/pki$ openssl x509 -in apiserver-etcd-client.crt -noout -issuer
issuer=CN = etcd-ca
vagrant@controlplane:/etc/kubernetes/pki$ openssl x509 -in apiserver-kubelet-client.crt -noout -issuer
issuer=CN = kubernetes
vagrant@controlplane:/etc/kubernetes/pki$ openssl x509 -in apiserver.crt -noout -issuer
issuer=CN = kubernetes
```

|                              |     |                      |       |            |
|------------------------------|-----|----------------------|-------|------------|
| apiserver.crt                | --- | server crt           | ----- | kubernetes |
| apiserver-kubelet-client.crt | --- | client crt (kubelet) | ----- | kubernetes |
| apiserver-etcd-client.crt    | --- | client crt (etcd)    | ----- | etcd-ca    |

[ **openssl -in <certificate file> -noout -issuer** ] is used to see the CA who signed the certificate.

API Server is accessed via *kubectl*, *kubelet*, *scheduler*, *controller manager*,

*external users* so it is accessed via

*DNS names, IP addresses, service names*

so the certificate must include all i.e.

*kubernetes, kubernetes.default, kubernetes.default.svc,*

*kubernetes.default.svc.cluster.local, cluster IP, node IP*

- ✦ That's why one OpenSSL configuration file has to be created to write the *aliases* (alternate names) and to be used in to create *apiserver.crt* certificate.

```
openssl x509 -req -in apiserver.csr \
 -CA ca.crt -CAkey ca.key \
 -out apiserver.crt \
 -extensions v3_req \
 -extfile openssl.cnf
```

```
[req]
req_extensions = v3_req
distinguished_name = req_distinguished_name

[v3_req]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names

[alt_names]
DNS.1 = kubernetes
DNS.2 = kubernetes.default
DNS.3 = kubernetes.default.svc
DNS.4 = kubernetes.default.svc.cluster.local
IP.1 = 10.96.0.1
IP.2 = 172.17.0.87
```

#### ➤ Server Certificate: *kubelet*

- ✦ *kubelet* will also have 2 kind of certificates, *client* and *server*.
- ✦ **kubelet.crt** and **kubelet.key** are the server certificate and key for kubelet.  
*kube-apiserver* talks to *kubelet server* to get the details about the pods and send the commands.
- ✦ For all the *kubelets* as client, they talk to API Server.  
So, for the API Server to know about the *kubelet*, it needs follow a specific naming convention.  
**system:node:<nodeName>** [ this should be the name in the *ssl certificate* ]  
for example: **system:node:node01**
- ✦ To verify this, open worker node or controlplane node (kubelet will be there in both if installed by *kubeadm*)
  - ✦ Execute **systemctl cat kubelet** [ because *kubelet* doesn't run inside a pod unlike other resources ]

and get the *config.yaml* path.

```
Environment="KUBELET_KUBECONFIG_ARGS=--bootstrap-kubeconfig=/etc/kubernetes/
Environment="KUBELET_CONFIG_ARGS=--config=/var/lib/kubelet/config.yaml"
This is a file that "kubeadm init" and "kubeadm join" generates at runtime
```

- In that path i.e. */var/lib/kubelet/pki* the certificates and keys will be present.

```
vagrant@node01:/var/lib/kubelet/pki$ pwd
/var/lib/kubelet/pki
vagrant@node01:/var/lib/kubelet/pki$ ls
kubelet-client-2026-04-30-15-26-19.pem kubelet-client-current.pem kubelet.crt kubelet.key
```

- You can see the *name* and *group* of this *kubelet-client-current.pem* by the command

*openssl x509 -in <file name> -noout -text* [it'll give full details]

*openssl x509 -in <file name> -noout -subject* [it'll give only subjects]

```
vagrant@node01:/var/lib/kubelet/pki$ sudo openssl x509 -in kubelet-client-current.pem -noout -subject
subject=O = system:nodes, CN = system:node:node01
```

*system:node:node01* [ node's name is *node01* ]

[ due to permission and all, use *sudo* with this command ]

- F

- F

-

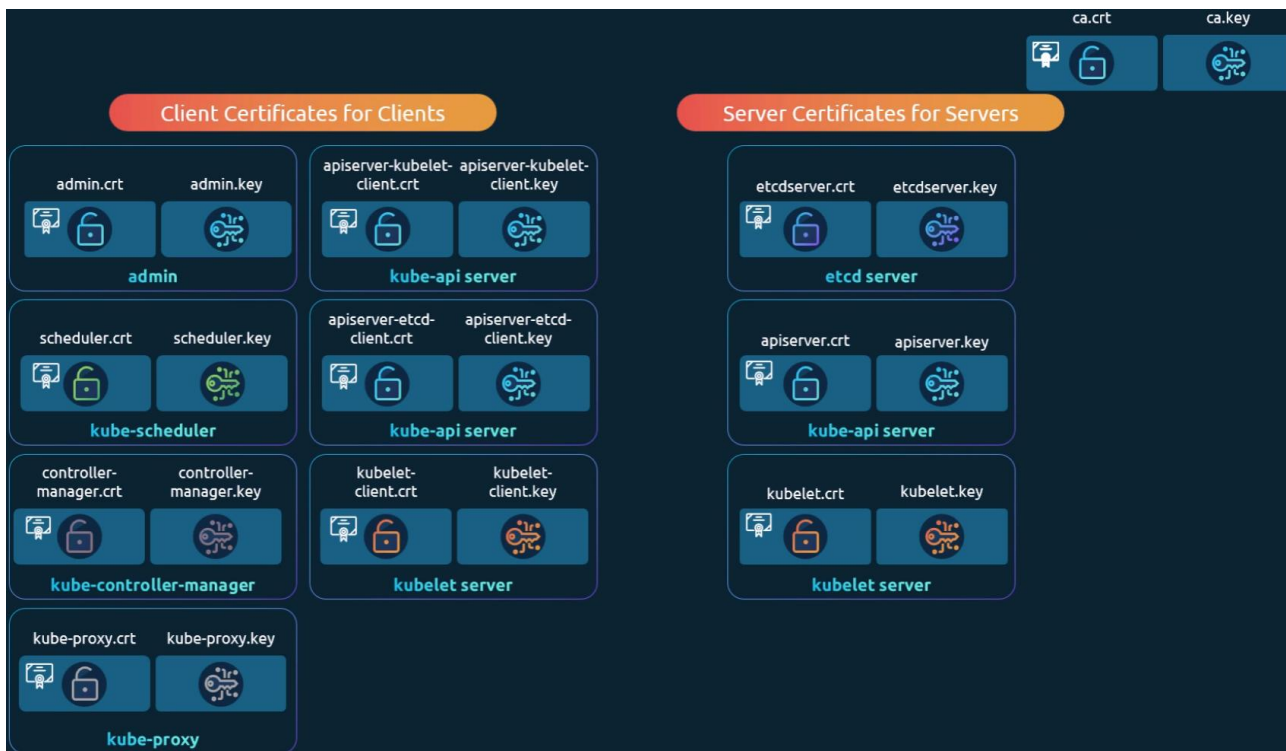


- If you want to see the details of the certificate, then execute this command

```
cat <crt file name> | openssl x509 -text -noout
```

```
vagrant@controlplane: /etc/kubernetes$ kubectl config view --raw -o json
Certificate:
 Data:
 Version: 3 (0x2)
 Serial Number: 3081854885164004134 (0x2ac4f2b08f80b326)
 Signature Algorithm: sha256WithRSAEncryption
 Issuer: CN = kubernetes
 Validity
 Not Before: Apr 30 15:01:07 2026 GMT
 Not After : Apr 30 15:06:07 2027 GMT
 Subject: O = kubeadm:cluster-admins, CN = kubernetes-admin
 Subject Public Key Info:
 Public Key Algorithm: rsaEncryption
 Public-Key: (2048 bit)
 Modulus:
 00:22:e8:cd:0c:fa:e0:00:1f:fd:f5:65:e5:8c:f7:
```

- The below is details about all the servers and clients.



- F

## LEGACY NOTES (JUST FOR MORE INFO)

➤ Now, we'll generate the client certificates.

➤ Admin User (kubectl) [ client certificate ]

- ⌘ Generate admin.key using the below command.

```
openssl genrsa -out admin.key 2048
```

- ⌘ Generate admin.csr using the below command

```
openssl req -new -key admin.key -subj "/CN=kube-admin" -out admin.csr
```

[ the name can be anything, but *kube-admin* is the name *kubectl* client authenticates with ]

if you want to group the keys and certificates (system, server ...etc)

```
openssl req -new -key admin.key -subj "/CN=kube-admin/O=system:masters" -out admin.csr
```

[ /O means *Organization* (group in Kubernetes) ]

here the group is **system:masters** , we want a *admin user*, not a normal user.

Keeping the group *system:masters* allow the user the admin privileges.

There is something called **RoleBinding**, in that the *user* is mapped. That

user's name must be same as /CN of the *certificate* otherwise it'll fail.

Inside the *config file*, the user's name can be different it doesn't create any issue.

```
users:
- name: system:kube-scheduler
 user:
 client-certificate-data: LS...
```

[ /etc/kubernetes/scheduler.conf ]

here the user's name can be different, no problem.

If you see the below, the group name of *cluster-admins* is **system:masters**

(that's why the group was added to the admin user)



## KubeConfig File

- To talk to API Server, you need to attach the *certificates* and all to authenticate yourself.

```
curl https://my-kube-playground:6443/api/v1/pods \
 --key admin.key \
 --cert admin.crt \
 --cacert ca.crt
```

Same for **kubectl** command as well, you have to provide these

```
kubectl get pods \
 --server my-kube-playground:6443 \
 --client-key admin.key \
 --client-certificate admin.crt \
 --certificate-authority ca.crt
```

- ⌘ So, rather than writing all these things every time, you can just add the details in *kubeconfig* file and just execute the *kubectl* command without providing all those keys and certificates.
- ⌘ If the *kubeconfig* file is present at *~/.kube/config* then *kubectl* automatically take that.

Otherwise, you need to use the flag *--kubeconfig* to provide the kubeconfig file's path.

```
kubectl get pods --kubeconfig config
```

- Structure of *kubeconfig* file
  - ⌘ **Clusters:** Represent various Kubernetes clusters (e.g., development, testing, production, or cloud-based clusters).
  - ⌘ **Users:** Contain credentials and client certificate information for users (e.g., admin, dev, prod).
  - ⌘ **Contexts:** Link clusters and users to define which user accesses which cluster. For example, a context named "admin@production" may indicate that the admin account is used to access the production cluster.  
It is not necessary that the name will be *<user name>@<cluster name>*, its just a CONVENTION.

- To change the current context

**kubectl config use-context** <context name> [it updates the config file's current-context field]

```
apiVersion: v1
kind: Config
clusters:
- name: my-kube-playground
 cluster:
 certificate-authority: ca.crt
 server: https://my-kube-playground:6443
contexts:
- name: my-kube-admin@my-kube-playground
 context:
 cluster: my-kube-playground
 user: my-kube-admin
users:
- name: my-kube-admin
 user:
 client-certificate: admin.crt
 client-key: admin.key
```

- Instead of the paths of certificates and key files, you can directly pass the content of those (crt & key) encoded in **base64** format.

```
cat ca.crt | base64 [command to encode to base64]
```

```
apiVersion: v1
clusters:
- cluster:
 certificate-authority-data: LS0tLS1CRUdJTiB1
 server: https://192.168.56.11:6443
 name: kubernetes
contexts:
- context:
 cluster: kubernetes
 user: kubernetes-admin
 name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
users:
- name: kubernetes-admin
 user:
 client-certificate-data: LS0tLS1CRUdJTiB1
 client-key-data: LS0tLS1CRUdJTiB1SU0EUGUfJ3
```

➤ To access the API Server using *curl* (direct http request) ---- **JUST FOR INFO**

- In my case I couldn't find the *admin.crt* and *admin.key* files, so I had to convert the *client-certificate-data* and *client-key-data* fields (present inside *~/.kube/config*) to *base64* (BEGIN, END) format.

```
kubectl config view --raw -o jsonpath='{.users[0].user.client-key-data}' |
base64 -d | sudo tee admin.key
```

[ *base64 -d* convert the content to *BEGIN, END* type of content ]

- Similarly extract the *client-cert-data* as well.
- Then just execute the *curl* command giving the *ca certificate, admin certificate, admin key* and it'll work.

```
curl https://192.168.56.11:6443 --cert admin.crt --key admin.key --cacert
ca.crt
```

[ If it doesn't work, then check the permissions of the keys and certificate files ]

[ NOTE: It is **https** not *http* ]

- The above base url (*https://<IP>:6443*) give all the available paths that *kube-apiserver* accepts.

```
{
 "paths": [
 "/.well-known/openid-configuration",
 "/api",
 "/api/v1",
 "/apis",
 "/apis/",
 "/apis/admissionregistration.k8s.io",
 "/apis/admissionregistration.k8s.io/v1",
 "/apis/apiextensions.k8s.io",
 "/apis/apiextensions.k8s.io/v1",
 "/apis/apiregistration.k8s.io",
 "/apis/apiregistration.k8s.io/v1",
 "/apis/apps",
 "/apis/apps/v1",
 "/apis/authentication.k8s.io",
 "/apis/authentication.k8s.io/v1"
```

- There are 2 ways you can talk to *kube-apiserver*, one with *kubectl* another is directly *http request*.

➤

# API Server

- When you trigger this http request

`curl https://192.168.56.11:6443 --cert admin.crt --key admin.key --cacert ca.crt`

you'll get the list of *api paths* of the kube apiserver.

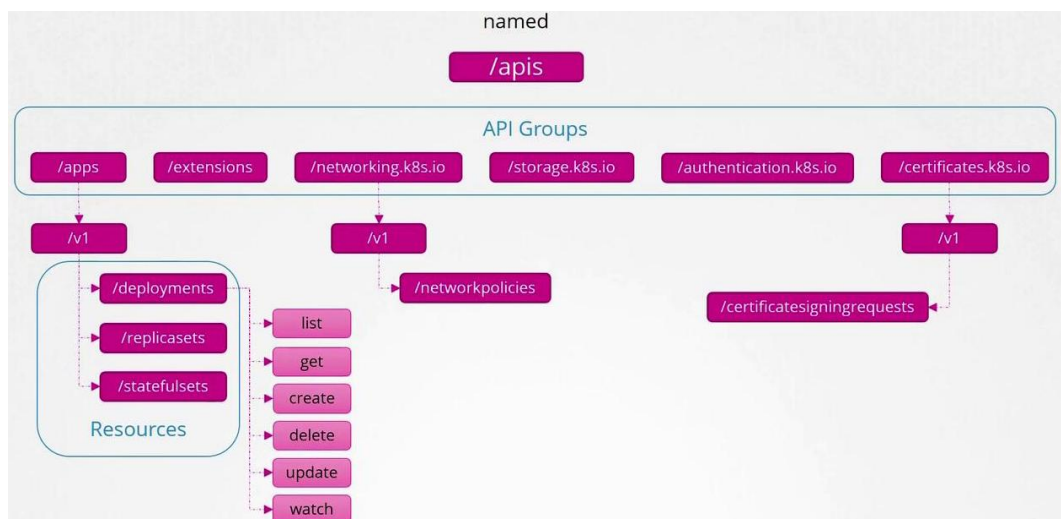
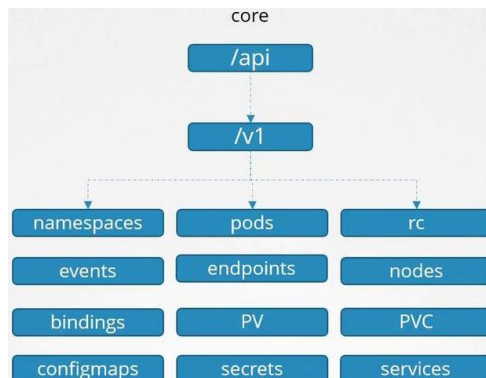
```
vagrant@controlplane:/etc/kubernetes/temp$ sudo curl https://controlplane:6443 --cert admin.crt --key admin.key --cacert ca.crt
{
 "paths": [
 "/.well-known/openid-configuration",
 "/api",
 "/api/v1",
 "/apis",
 "/apis/",
 "/apis/admissionregistration.k8s.io",
 "/apis/admissionregistration.k8s.io/v1",
 "/apis/apixtensions.k8s.io"
```



- The cluster functionality is divided in 2 major categories.

`/api` : Core Api root Path

`/apis` : Named Api root Path



- ⌘ Whatever things will be added, those will be added in *named* API groups because there are properly organized.  
`/api` is the core API Group.
  - ⌘ `/apis` is named API root path because it contains *API groups*.  
 where `/api` doesn't contain any *API group*, rather contains the *version* directly.
- When you write the *yaml* file, you write either "`v1`" or "`<api group name>/<version>`" like this.
  - ⌘ If you write only version i.e. "`v1`" then it'll take the core API root path [`/api`], because it doesn't contain any API group.  
`/api/v1/pod`, `/api/v1/services` ...etc
  - ⌘ If you write something like "`apps/v1`" then it'll take the Named API root path [`/apis`] because it contains the groups.  
`/apis/apps/v1/replicaset`, ..etc
- Names of the components:
  - ⌘ `/apps`, `/extensions` ...etc present inside `/apis` (Named API groups) are called **API Groups**.
  - ⌘ Inside those **Versions** are there (`v1`).
  - ⌘ Inside each version, **Resources** are there.  
 Ex: `apps/v1` contains *deployments*, *replicasets*, *statefulsets* ...etc.
  - ⌘ Each Resources contains **Verbs** like *list*, *create*, *delete* ..etc  
 ex: *list* the deployments, *create* deployment, *delete* deployment ..etc.
- To see all the named API groups, execute this command  
`curl https://controlplane:6443/apis -k --cert admin.crt --key admin.key --cacert ca.crt | grep name` [ NOTE: *admin.crt*, *admin.key*, *ca.crt* should be given ]

```
9530 0 0930 0 0 84087
"name": "apiregistration.k8s.io",
"name": "apps",
"name": "events.k8s.io",
"name": "authentication.k8s.io",
"name": "authorization.k8s.io",
"name": "autoscaling",
"name": "batch",
"name": "certificates.k8s.io",
"name": "networking.k8s.io",
"name": "policy",
"name": "rbac.authorization.k8s.io",
"name": "storage.k8s.io",
"name": "admissionregistration.k8s.io",
"name": "apiextensions.k8s.io",
"name": "scheduling.k8s.io",
"name": "coordination.k8s.io",
"name": "node.k8s.io",
"name": "discovery.k8s.io",
"name": "resource.k8s.io",
"name": "flowcontrol.apiserver.k8s.io",
"name": "projectcalico.org",
"name": "crd.projectcalico.org",
"name": "operator.tigera.io",
"name": "policy.networking.k8s.io",
+Operator 1.7.0 (https://kubernetes.io/docs/
```

^

^

# Authorization

- There are multiple types of Authorization mechanisms

- ⌘ Node
- ⌘ ABAC [ Attribute Based Access Control ]
- ⌘ RBAC [ Role Based Access Control ]
- ⌘ Webhook

- Node Authorizer

- ⌘ We saw before, the *kubelet* certificate's name was in the form of

**system:node:<node name>**

- ⌘ This Node Authorizer is an authorization method of *kube-apiserver* that grant permissions to *kubelets* (nodes), limit their access to the resources linked with their *respective node only*, prevents nodes from other node's or pod's data.
- ⌘ If you see the *subject* of the certificate of *kubelet* in any node, you can see the CN and group

```
vagrant@node01:/var/lib/kubelet/pki$ ls
kubelet-client-2026-04-30-15-26-19.pem kubelet-client-current.pem kubelet.crt kubelet.key
vagrant@node01:/var/lib/kubelet/pki$ sudo openssl x509 -in kubelet-client-current.pem -noout -subject
subject=O = system:nodes, CN = system:node:node01
```

[ CN: system:node:node01, O: system:nodes ]

[ node01 is then worker node's name; O means group, CN means common name or name ]

- ⌘ This certificate is signed by Kubernetes itself.

```
vagrant@node01:/var/lib/kubelet/pki$ sudo openssl x509 -in kubelet-client-current.pem -noout -issuer
issuer=CN = kubernetes
```

- ⌘ You **CAN'T CHANGE** this format of **system:node:<node name>**

It is HARD-CODED inside Kubernetes.

- ⌘ This method is enforced by *kube-apiserver* ; it expects the CN and group in that format.

If the format is not proper, then the request is treated as a *normal user*, **NOT a Node**.

- ⌘ If you see the *kube-apiserver.yaml* (Kube Api Server's pod config yaml file) [ */etc/kubernetes/manifests/kube-apiserver.yaml* ] then you can see the *--authorization-mode* flag would have set to **Node** along with some other authorization mode.



[ if kube-apiserver is running as a process, not Pod, then you can see inside *service* file using `systemctl cat kube-apiserver` ]

```
spec:
 containers:
 - command:
 - kube-apiserver
 - --advertise-address=192.168.56.11
 - --allow-privileged=true
 - --authorization-mode=Node,RBAC
 client_ca_file: /etc/kubernetes/rlk
```

### ➤ ABAC [ Attribute Based Access Control ]

- ⌘ You provide access to a set of users or group a specific set of permissions.
- ⌘ In here, you create json file of *kind: Policy* and provide that to *kube-apiserver*

```
{ "kind": "Policy", "spec": { "user": "dev-user", "namespace": "*", "resource": "pods", "apiGroup": "*" } }
{ "kind": "Policy", "spec": { "user": "dev-user-2", "namespace": "*", "resource": "pods", "apiGroup": "*" } }
{ "kind": "Policy", "spec": { "group": "dev-users", "namespace": "*", "resource": "pods", "apiGroup": "*" } }
```

- ⌘ When a request hits the kube-apiserver, its attributes are extracted:  
*user, verb, resources, namespace*  
[ *alok, get, pods, default* (For Example)]  
and apiserver checks ABAC Policy file; if match found then *allow* else *reject*.
- ⌘ It is **not-preferable** because for each change you need to update the file for each user/ group.
- ⌘ **NOTE:** ABAC policy file is not a valid JSON format. In json, multiple root level objects without being part of an array cannot be present. But for Kubernetes, it is valid.

### ➤ RBAC [ Role Based Access Control ]

- ⌘ Instead of writing rules in a file (like ABAC), RBAC uses *Kubernetes Objects* [just like pods, replicaset ..etc; you can see in `kubectrl api-resources` ], you can write *yaml* and create the objects.
- ⌘ The object that RBAC provides are

**Role, RoleBinding, ClusterRole, ClusterRoleBinding**

```
vagrant@controlplane:/etc/kubernetes/manifests$ kubectl api-resources | grep -i role
clusterrolebindings rba
clusterroles rba
rolebindings rba
roles rba
```

- ⌘ Its kind of, create a role, and associate users/groups to that role.  
If you want to update any permissions, then **only need to change in the Role object** unlike ABAC where you need to update each Policy.
- ⌘ Role: permissions within a *single namespace*.  
ClusterRole: permissions across the whole *cluster* or cluster level resources (like nodes)
  - ⌘ Give access to pod in a namespace: *Role*  
Give access to pod in ALL namespaces: *ClusterRole*  
Give access to nodes: *ClusterRole*
  - ⌘ **all things that can be done using role can also be done with clusterrole.**
- ⌘ RoleBinding: grant access within a *specific namespace*.  
ClusterRoleBinding: grant access across the *entire cluster*.
  - ⌘ **RoleBinding:**  
**binds a user/group to Role, or ClusterRole**  
**scope: one namespace only**
  - ⌘ **ClusterRoleBinding:**  
**binds a user/group to ClusterRole ONLY**  
**scope: entire cluster**
- **Webhook**
  - ⌘ It'll make Kubernetes to query an third party REST service to get to know if the user is authorized for that specific request or not.
  - ⌘ One third party provider is: *Open Policy Client*
- There are 2 more: *AlwaysAllow, AlwaysDeny*
  - ⌘ When no authorization mechanism is provided, by default it is *AlwaysAllow*.

```
ExecStart=/usr/local/bin/kube-apiserver \
 --advertise-address=${INTERNAL_IP} \
 --allow-privileged=true \
 --apiserver-count=3 \
 --authorization-mode=AlwaysAllow \
 bind-address=0.0.0.0 \
```

- ⌘ You can give multiple authorization mechanism here (comma separated)

```
ExecStart=/usr/local/bin/kube-apiserver \
 --advertise-address=${INTERNAL_IP} \
 --allow-privileged=true \
 --apiserver-count=3 \
 --authorization-mode=Node,RBAC,Webhook \
```

- ⌘ When multiple authorization mechanism is given, the execution happens sequentially (from left to right)

in above case,

first *Node* authorization will happen;

if it rejects, then RBAC will happen;

if it rejects, then Webhook will happen.

- ⌘ It'll keep trying till the authorization is allowed.

If it is allowed, then it'll no go to the next authorization mechanism to the right.

- ⌘ If all the authorization reject the request, then user will get forbidden.

- To check if you are authorized for doing something, then use **auth can-i** like the below

```
vagrant@controlplane:~$ kubectl auth can-i create deployments
yes
```

- If you want to check the authorization of another user, then use **--as <user name>**

```
vagrant@controlplane:~$ kubectl auth can-i create deployments --as dev-user
no
```

- F
- F
- F
- F
-

## RBAC

### ➤ Role

- ⌘ It is present in “`rbac.authorization.k8s.io/v1`”
- ⌘ To create a role, you need to write the rules.
- ⌘ Each rule contains *apiGroups*, *resources*, *verbs*

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: pod-reader
rules:
- apiGroups:
 - ""
 resources:
 - pods
 verbs:
 - get
 - list
 - watch
```

- ⌘ **NOTE:** apiGroups `""` doesn't mean all api groups, it means Core Api Group (because in core api group is not there)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: developer
rules:
- apiGroups: [""]
 resources: ["pods"]
 verbs: ["list", "get", "create", "update", "delete"]
- apiGroups: [""]
 resources: ["ConfigMap"]
 verbs: ["create"]
```

- ⌘ The Role is *namespace* scoped.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 namespace: default
 name: pod-reader
rules:
- apiGroups: [""] # "" indicates the core API group
 resources: ["pods"]
 verbs: ["get", "watch", "list"]
```

[ namespace is provided here ]

- ⌘ Lets say in the *namespace* so many pods are there, but you just want to give access of some specific pods in that role, other pods should not be accessible. In that case, you can use *resourceNames* field.

```
kind: Role
metadata:
 name: developer
rules:
- apiGroups: [""]
 resources: ["pods"]
 verbs: ["get", "create", "update"]
 resourceNames: ["blue", "orange"]
```

### ➤ RoleBinding

- ⌘ This is to bind a Role to a User/Group.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: devuser-developer-binding
subjects:
- kind: User
 name: dev-user
 apiGroup: rbac.authorization.k8s.io
roleRef:
 kind: Role
 name: developer
 apiGroup: rbac.authorization.k8s.io
```

- ⌘ This is also specific to *namespace*.

### ⌘ NOTE ----- IMPORTANT -----

here you can see **kind: User** [ there is nothing called User or Group object (you'll not find this when *kubectl api-resources* is executed.) ]

here, this **subjects[\*].kind** is not same as **.kind** (top level); it is just a key to define what kind of subject is that.

Now, you might be thinking, **apiGroup** is not needed because the *apiGroup* `rbac.authorization.k8s.io` doesn't contain anything called *User* or *Group* because if it contains, then it would be a Kubernetes Object (like pod, replicaset, role, clusterrole, ...etc )

kind can be *User*, *Group* or *ServiceAccount* ; and *ServiceAccount* is a real object (present in Core API Group) but *User* and *Group* are not object.

The RBAC matching logic is written using the combination of **kind** and **apiGroup**, so if you don't give *apiGroup* it'll fail.

```
(kind + apiGroup)
```

it'll be always

```
apiGroup: rbac.authorization.k8s.io
```

 (for User, Group) or

```
apiGroup: ""
```

 (for ServiceAccount)

```

apiVersion: rbac.authorization.k8s.io/v1
This role binding allows "jane" to read pods in the "default" namespace.
You need to already have a Role named "pod-reader" in that namespace.
kind: RoleBinding
metadata:
 name: read-pods
 namespace: default
subjects:
You can specify more than one "subject"
- kind: User
 name: jane # "name" is case sensitive
 apiGroup: rbac.authorization.k8s.io
roleRef:
 # "roleRef" specifies the binding to a Role / ClusterRole
 kind: Role #this must be Role or ClusterRole
 name: pod-reader # this must match the name of the Role or ClusterRole you wish to bind to
 apiGroup: rbac.authorization.k8s.io
```

- One confusion is **namespace**, if we have given *namespace* field inside **Role**, then why do we need to give it again in **RoleBinding**?

**namespace** in **Role** defines "In which namespace the Role should be created"

**namespace** in **RoleBinding** defines "I'll bind to the role present in that namespace"

so basically, if you have given *namespace: ns1* in the **RoleBinding**, then the specific role must be present in the *namespace ns1*. if the role is present in some other namespace then it'll not work.

[ its only for **Role**. Not for **ClusterRole** ]



**Role** and **RoleBinding** are *namespace scoped*.

If *namespace* is not provided, then the objects (*Role*, *RoleBinding*) will be created in the same *namespace* where they are being called.

Not the ~~default~~ *namespace*.

### ➤ ClusterRole

- It is same as **Role**, but it is not limited to any *namespace*.

Its *cluster scoped*.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 # "namespace" omitted since ClusterRoles are not namespaced
 name: secret-reader
rules:
- apiGroups: [""]
 #
 # at the HTTP level, the name of the resource for accessing Secret
 # objects is "secrets"
 resources: ["secrets"]
 verbs: ["get", "watch", "list"]
```

unlike **Role**, *namespace* field will not be here, because it'll be available *cluster wide*.

- One more difference between **Role** and **ClusterRole** is:

**Role** can have *namespace* resources.

**ClusterRole** can have both *namespace* and *cluster-scoped* resources.

- To check namespace and non-namespaced resources, execute the command

**kubectl api-resources --namespaced=true** [namespaced resources]

**kubectl api-resources --namespaced=false** [non-namespaced resources]

```
vagrant@controlplane:~$ kubectl api-resources --namespaced=false
NAME SHORTNAMES APIVERSION
componentstatuses cs v1
namespaces ns v1
nodes no v1
persistentvolumes pv v1
mutatingadmissionpolicies admn v1
mutatingadmissionpolicybindings admn v1
mutatingwebhookconfigurations admn v1
validatingadmissionpolicies admn v1
```

### ➤ ClusterRoleBinding

- It is used to bind the **ClusterRole** to any User or Group or ServiceAccount.



- ⌘ If you bind a *ClusterRole* using *ClusterRoleBinding* then the authorized resources (mentioned in *ClusterRole*) will be authorized to the User or Group or ServiceAccount with which the *ClusterRole* is bound to.
- ⌘ *ClusterRole* can be bound with *RoleBinding* as well, in this case the *authorization* will be applied to a specific namespace where *RoleBinding* will be created.
- After creating a binding, if you want to change the *roleRef* you'll get a validation error.

If you really want to do it, then you need to delete the *binding* object and create a new one with the new *roleRef*.
-

## ServiceAccount

- There are 2 types of accounts: **User, Service**.
  - ⌘ User account is used by the users like admin, developer.
  - ⌘ Service account is used by applications to interact with kubernetes cluster like *Jenkins, Prometheus* ..etc.
    - [ Prometheus pulls kubernetes api for the performance matrix ]
    - [ Jenkins used Service Account to deploy the application in Kubernetes cluster ]
- In all namespace, there will be a default *ServiceAccount* whose name is “**default**”.

```
calico-apiserver default 6d5h
calico-system default 6d5h
default default 6d5h
kube-node-lease default 6d5h
kube-public default 6d5h
kube-system default 6d5h
test default 5d
tigera-operator default 6d5h
```

[ namespace    ServiceAccount    AGE]    (columns)

- ⌘ When a pod is created, by-default the *ServiceAccount* present in that namespace (where pod is created) will be attached to the pod *IF NOT EXPLICITLY MENTIONED*.

```
vagrant@controlplane:~/k8s$ kubectl describe pods dev-pod
Name: dev-pod
Namespace: default
Priority: 0
Service Account: default
```

- ⌘ **Pod needs the ServiceAccount to talk to API Server.**
    - [ there are multiple reasons for which Pod talks to API Server.
    - But, ~~pod to pod communication doesn't happen through API Server~~ ]
  - Both *internal components* (like pods, ..etc) and *external services* (like Jenkins ..etc) uses **ServiceAccount** to communicate with API Server.
    - ⌘ ServiceAccount is required to give access to the resources via **Role** or **ClusterRole**.
    - ⌘ For external services, you need to create a token linked to the ServiceAccount.
      - With that token (Authorization Bearer) the external service can communicate to API server.
- kubectl create token <service account name>**

```
kubectl create token external-sa -n dev
```

NOTE: Token **doesn't have name**, and it is **not stored anywhere**.

When you execute this command, a token will be generated and displayed in the screen using which the Service can trigger REST request using

**Bearer <generated token>**.

- ⌘ For internal components (like Pod), the *Service Account* is directly attached to it. But that doesn't mean it doesn't require token.

For internal components, a **token is generated automatically** using which they can talk to API Server.

- ⌘ When a *Service Account* is attached to a Pod, kubernetes creates a virtual volume and mounts that to the Pod. [ if

**spec.automountServiceAccountToken: true** inside Pod

```
apiVersion: v1
kind: Pod
metadata:
 name: my-kubernetes-dashboard
spec:
 containers:
 - name: my-kubernetes-dashboard
 image: my-kubernetes-dashboard
 automountServiceAccountToken: false
```

(or)

**automountServiceAccountToken: true** inside ServiceAccount]

This mount path is: [/var/run/secrets/kubernetes.io/serviceaccount/token](#)

[ inside Pod ]

```
Mounts:
/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-rdvhr (ro)
```

this contains *token*, *ca.crt*, *namespace* details.

[ NOTE: volume's lifecycle = pod's life cycle; **if pod is gone, volume is also destroyed** ]

- ⌘ In short,

ServiceAccount is required to provide specific authorization in Role or ClusterRole.

After that, a token will be generated linked to that Service Account (automatic for internal components, manually for external services) and that

token is not stored anywhere.

For Pods, the token is generated and stored inside that mounted path and it gets refreshed when it is expired.

Virtual volume's is destroyed once pod is gone.

## Image Security

- When you provide the image name, it actually translates that to **library/<image name>** because it is where the official images are present in docker hub.

```
image: library/nginx
```

- It pulls from Docker Registry (Docker Hub) by default,

```
image: docker.io/library/nginx
```

- But, if you are having a private container registry and want to pull the image from there, then you need to give full path against the *image* key.

```
image: private-registry.io/apps/internal-app
```

- Now, as this is private registry and it needs to be authenticated to access that, you need to create a resource called **secret**.

```
kubectl create secret docker-registry regcred \
 --docker-server=private-registry.io \
 --docker-username=registry-user \
 --docker-password=registry-password \
 --docker-email=registry-user@org.com
```

docker-registry: type of secret

regcred: name of secret (just an identifier)

- And mention that *secret* in the Pod's yaml file inside the field

*spec.imagePullSecrets*

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx-pod
spec:
 containers:
 - name: nginx
 image: private-registry.io/apps/internal-app
 imagePullSecrets:
 - name: regcred
```

## Security Context

- In docker, while creating a container, you can specify if you don't want the **root** user as default.

[ root is risky, if Pod is compromised, then it'll have full access because of root user ]

you can specify **--user=<user id>** while running Docker container.

```
docker run --user=1001 ubuntu
```

- When a container runs, it doesn't get full root privileges *even if it is root user*.

Docker uses a Linux feature called **capabilities** to split root power into smaller pieces.

Using **--cap-add <capability name>** you can provide the specific capability to the user.

```
docker run --cap-add MAC_ADMIN ubuntu
```

```
NET_ADMIN → control networking
SYS_ADMIN → system-level control
CHOWN → change file ownership
MAC_ADMIN → control security policies
```

- In Kubernetes also you can configure all of these.

- ⌘ As, Containers run inside a Pod in Kubernetes.

So, if you configure the security settings at Pod level, it'll be **applied to all the containers** running inside it.

If you configure the security settings at both Pod and Container level, Pod's security settings will be **overridden by Container's security settings**.

- ⌘ You can add a field **spec.containers[\*].securityContext** to specify the security configurations *at container level*.

```
apiVersion: v1
kind: Pod
metadata:
 name: web-pod
spec:
 containers:
 - name: ubuntu
 image: ubuntu
 command: ["sleep", "3600"]
 securityContext:
 runAsUser: 1000
 capabilities:
 add: ["MAC_ADMIN"]
```

[ Container Level ]

- ⌘ If you want to configure it in *pod level* then, add the field

*spec.securityContext* .

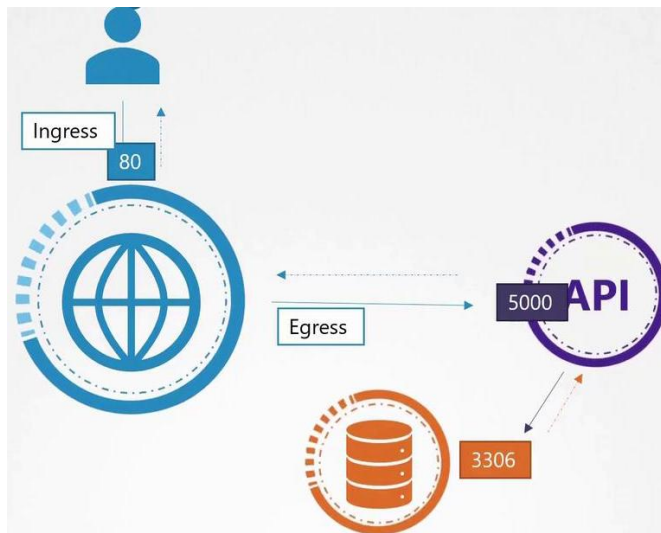
```
apiVersion: v1
kind: Pod
metadata:
 name: web-pod
spec:
 securityContext:
 runAsUser: 1000
 capabilities:
 add: ["MAC_ADMIN"]
 containers:
```

- F



## Network Policies

- In Kubernetes, there is no restriction for inter Pod communication by default. Any pod can talk to any other pod using the target Pod's service (ClusterIP).
  - ⌘ To restrict this *Network Policies* are used.
  - ⌘ Its kind of ABAC (Attribute Based Access Control), because here also Policy was there which had to be written in a JSON format.  
[ Not same as ABAC, because ABAC is of course used for Authorization; and Network Policy is used for restrict the access of Pods ]
  - ⌘ For example, there are 3 types of pods running.
    - ⌘ *Frontend, Backend, DB*
    - ⌘ By default, Frontend can access the DB pods which should not happen.
- Consider the below image



- ⌘ Web Server:
    - Ingress: accept HTTP traffic on Port 80
    - Egress: sends traffic on port 5000 to Backend API server.
  - ⌘ Backend API Server
    - Ingress: Port 5000
    - Egress: Port 3306 to DB
  - ⌘ DB
    - Ingress: Port 3306
- F
  - F
  -

- securityContext,
- What field can you use to allow access to specific resources alone in Kubernetes
- F
- F
- F
- F
-

## Important Points

- Try to use *imperative commands* as much as you can in Exam.
- If you forget the keywords and resources, then execute **kubectl api-resources** you'll get the keywords.

```
vagrant@controlplane:~$ kubectl api-resources
```

| NAME                            | SHORTNAMES | APIVERSION                      | NAMESPACED | KIND                           |
|---------------------------------|------------|---------------------------------|------------|--------------------------------|
| bindings                        |            | v1                              | true       | Binding                        |
| componentstatuses               | cs         | v1                              | false      | ComponentStatus                |
| configmaps                      | cm         | v1                              | true       | ConfigMap                      |
| endpoints                       | ep         | v1                              | true       | Endpoints                      |
| events                          | ev         | v1                              | true       | Event                          |
| limitranges                     | limits     | v1                              | true       | LimitRange                     |
| namespaces                      | ns         | v1                              | false      | Namespace                      |
| nodes                           | no         | v1                              | false      | Node                           |
| persistentvolumeclaims          | pvc        | v1                              | true       | PersistentVolumeClaim          |
| persistentvolumes               | pv         | v1                              | false      | PersistentVolume               |
| Pods                            | po         | v1                              | true       | Pod                            |
| podtemplates                    |            | v1                              | true       | PodTemplate                    |
| replicationcontrollers          | rc         | v1                              | true       | ReplicationController          |
| resourcequotas                  | quota      | v1                              | true       | ResourceQuota                  |
| secrets                         |            | v1                              | true       | Secret                         |
| serviceaccounts                 | sa         | v1                              | true       | ServiceAccount                 |
| services                        | svc        | v1                              | true       | Service                        |
| mutatingadmissionpolicies       |            | admissionregistration.k8s.io/v1 | false      | MutatingAdmissionPolicy        |
| mutatingadmissionpolicybindings |            | admissionregistration.k8s.io/v1 | false      | MutatingAdmissionPolicyBinding |
| mutatingwebhookconfigurations   |            | admissionregistration.k8s.io/v1 | false      | MutatingWebhookConfiguration   |

[ you can see there is a column 'KIND' which you can directly write in the yaml ]

- ✧ **Shortnames** are also there to use in *imperative commands* which will SAVE TIME.
- If you want to get a documentation regarding any specific resource, you can use **kubectl explain <resource name>** command.

```
vagrant@controlplane:~$ kubectl explain pods
```

```
KIND: Pod
VERSION: v1
```

DESCRIPTION:

Pod is a collection of containers that can run on a host. This resource is created by clients and scheduled onto hosts.

FIELDS:

apiVersion <string>  
APIVersion defines the versioned schema of this representation of an object. Servers should convert recognized schemas to the latest internal value, and may reject unrecognized values. More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources>

kind <string>  
Kind is a string value representing the REST resource this object represents. Servers may infer this from the endpoint the client submits requests to. Cannot be updated. In CamelCase. More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds>

metadata <ObjectMeta>  
Standard object's metadata. More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata>

spec <PodSpec>  
Specification of the desired behavior of the pod. More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>

status <PodStatus>  
Most recently observed status of the pod. This data may not be up to date. Populated by the system. Read-only. More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>

- ♣ To go deeper into the specific fields of that resources,

**kubectl explain <resource name>.<field name>**

[ you can nest these any number times:

**resource.field.sub\_field.sub\_sub\_field .... etc ]**

```
vagrant@controlplane:~$ kubectl explain pods.spec
KIND: Pod
VERSION: v1

FIELD: spec <PodSpec>

DESCRIPTION:
 Specification of the desired behavior of the pod. More info:
 https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status
 PodSpec is a description of a pod.

FIELDS:
 activeDeadlineSeconds <integer>
 Optional duration in seconds the pod may be active on the node relative to
```

If you

want, more comprehensive, then use

**kubectl explain <resource name> --recursive**

```
vagrant@controlplane:~$ kubectl explain pods --recursive
KIND: Pod
VERSION: v1

DESCRIPTION:
 Pod is a collection of containers that can run on a host. The pod is the
 minimum unit of scheduling created by clients and scheduled onto hosts.

FIELDS:
 apiVersion <string>
 kind <string>
 metadata <ObjectMeta>
 annotations <map[string]string>
 creationTimestamp <string>
 deletionGracePeriodSeconds <integer>
```

- To get a basic yaml file instead of writing from scratch, use **--dry-run=client -o yaml** with the command.

[ with **run**, **create** for static] [ with **get** if the object is already running ]

**kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml**

**kubectl run my-pod --image=nginx --dry-run=client -o yaml**

```
vagrant@controlplane:~$ kubectl run my-pod --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
 labels:
 run: my-pod
 name: my-pod
spec:
 containers:
 - image: nginx
 name: my-pod
 resources: {}
 dnsPolicy: ClusterFirst
 restartPolicy: Always
```

```
vagrant@controlplane:~$ kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 labels:
 app: my-dep
 name: my-dep
spec:
 replicas: 1
 selector:
 matchLabels:
 app: my-dep
 strategy: {}
 template:
 metadata:
 labels:
 app: my-dep
 spec:
 containers:
 - image: nginx
 name: nginx
 resources: {}
```

- *config* file is present in `~/.kube/config` which contains all the details about *users*, *clusters*, *contexts*.
  - ⌘ *cluster* contains the list of *kube-api-server endpoint & certificate* (for identity).
  - ⌘ *users* contains the list of users each having a *unique name, key, certificate*.
  - ⌘ *context* contains the combination of triplet i.e. *user, cluster, namespace*.  
[ default namespace ]
  - ⌘ When you switch a context, the user will be updated, the cluster will be updated, and the default namespace will be updated.  
You can override the default namespace with the flag `--namespace=<ns name>`
  - ⌘ [kubectl config --help](#)  
command you can use to see the usage of different fields of configs.
- TAIN, TOLERATIONS, NODE SELECTOR, NODE AFFINITY,
  - ⌘ *taint* is to prevent a pod to be scheduled on a node (taint is applied on node).

- ⌘ *toleration* is to make a pod tolerant to get scheduled to a *taint* pod (toleration is applied on pod).
- ⌘ *nodeSelector* is used to assign a pod to a node only if the selector labels matches with the node's labels. [ HARD selection, if not matches then it is not scheduled ].
- ⌘ *nodeAffinity* is advanced version of *nodeSelector*, here *hard* and *soft* selection can be there with advanced selection queries.
- ⌘ Sometimes you need to use both the things *taint*, *toleration* and *nodeAffinity* to solve the real-world problem.
- You can give *requests* and *limits* on the pods.
  - ⌘ *requests* means the pod's minimum requirement (cpu and memory) to be scheduled to a node.
  - ⌘ *limits* means the pod can use that much amount of resources at max.
    - ⌘ If *cpu* usage exceeds the *limit*, then it *slows down*.
    - ⌘ If *memory* usage exceeds, then pod is *killed*.
  - ⌘ Instead of giving this *requests* and *limits* in all the pods, you can create *LimitRange* (kind) which will contain the *request*, *limits*, *max*, *min* .
    - ⌘ If any pod doesn't have the request and limit fields, then it'll inject its default values.
  - ⌘ *LimitRange* works at *namespace* level.
- *DaemonSet* is same as *ReplicaSet*, it just create a replica of the pod in each worker node.
- *kubeadm* creates static pods on control plane nodes to run the controlplane resources.
- API Server stores the details of all the pods in *etcd* (*normal pods* or *static pods*). in the annotation 2 extra fields are there in case of *static pods* which differentiate it from the normal pods.
  - ⌘ When you edit a pod using *kubectl edit* command, it updates *etcd* and send *kubelet* (in streaming connection) the new *PodSpec* then *kubelet* compares it with the existing *PodSpec*.  
Then changes if allowed otherwise reject.  
In each changes, *containerd* creates new container and the container ID is changed.

⌘ In case of *static pods*, you can't change using *kubectrl edit* command in control plane nodes.

You need to update the *yaml* file inside the *manifest* directory of the target node (where the static pod is running).

➤ READ THE THEORY HAVING HEADING "*Resources inside Pods vs Resources as normal Processes*"; DON'T SKIP IT.

➤

➤ F

➤ F

➤ F